Sales Prediction for Big Mart Outlets

The BigMart Sales project focuses on analyzing retail data to understand the key factors influencing product sales across multiple outlets and to develop predictive and analytical insights. The dataset consists of product-level and outlet-level attributes such as item price (MRP), visibility, category, outlet type, and sales.This calls for brushing up my EDA skills and also makes me think deeper on various features and imputations.

```
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings;
warnings.filterwarnings('ignore')
sns.set_style("white")
%matplotlib inline

#Reading the training and testing dataset

df = pd.read_csv('./data/train_bigmart.csv')
df_test = pd.read_csv('./data/test_bigmart.csv')

df_copy = df.copy()
df_test_copy = df_test.copy()
df.head()
```

```
  Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
0           FDA15         9.30          Low Fat         0.016047
1           DRC01         5.92          Regular         0.019278
2           FDN15        17.50          Low Fat         0.016760
3           FDX07        19.20          Regular         0.000000
4           NCD19         8.93          Low Fat         0.000000

                Item_Type  Item_MRP Outlet_Identifier  \
0                   Dairy  249.8092            OUT049
1             Soft Drinks   48.2692            OUT018
2                    Meat  141.6180            OUT049
3  Fruits and Vegetables  182.0950            OUT010
4               Household   53.8614            OUT013

   Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  \
0                       1999      Medium               Tier 1
1                       2009      Medium               Tier 3
2                       1999      Medium               Tier 1
3                       1998         NaN               Tier 3
```

```
4                       1987        High               Tier 3

         Outlet_Type  Item_Outlet_Sales
0  Supermarket Type1          3735.1380
1  Supermarket Type2           443.4228
2  Supermarket Type1          2097.2700
3      Grocery Store           732.3800
4  Supermarket Type1           994.7052

print(f"{df.shape = }, {df_test.shape=}")

df.shape = (8523, 12), df_test.shape=(5681, 11)

df.info()

<class 'pandas.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Item_Identifier            8523 non-null   str
 1   Item_Weight                7060 non-null   float64
 2   Item_Fat_Content           8523 non-null   str
 3   Item_Visibility            8523 non-null   float64
 4   Item_Type                  8523 non-null   str
 5   Item_MRP                   8523 non-null   float64
 6   Outlet_Identifier          8523 non-null   str
 7   Outlet_Establishment_Year  8523 non-null   int64
 8   Outlet_Size                6113 non-null   str
 9   Outlet_Location_Type       8523 non-null   str
 10  Outlet_Type                8523 non-null   str
 11  Item_Outlet_Sales          8523 non-null   float64
dtypes: float64(4), int64(1), str(7)
memory usage: 1.2 MB

df.isna().sum()

Item_Identifier                 0
Item_Weight                  1463
Item_Fat_Content                0
Item_Visibility                 0
Item_Type                       0
Item_MRP                        0
Outlet_Identifier               0
Outlet_Establishment_Year       0
Outlet_Size                  2410
Outlet_Location_Type            0
Outlet_Type                     0
Item_Outlet_Sales               0
dtype: int64
```

```
df.describe()

       Item_Weight  Item_Visibility     Item_MRP
Outlet_Establishment_Year  \
count  7060.000000      8523.000000  8523.000000
8523.000000
mean     12.857645         0.066132   140.992782
1997.831867
std       4.643456         0.051598    62.275067
8.371760
min       4.555000         0.000000    31.290000
1985.000000
25%       8.773750         0.026989    93.826500
1987.000000
50%      12.600000         0.053931   143.012800
1999.000000
75%      16.850000         0.094585   185.643700
2004.000000
max      21.350000         0.328391   266.888400
2009.000000

       Item_Outlet_Sales
count        8523.000000
mean         2181.288914
std          1706.499616
min            33.290000
25%           834.247400
50%          1794.331000
75%          3101.296400
max         13086.964800
```

Observations:

1. Item_Weight — Only 7,060 of 8,523 rows have values (~1,463 missing, ~17%). Range (4.56–21.35) and mean/std look reasonable, so missing values likely need imputation (e.g., by item identifier/type group mean).
2. Item_Visibility — Minimum is 0.0, which is physically implausible (every shelved item has some visibility). This is likely a data-entry artifact / disguised missing value and should be treated (e.g., replace 0 with NaN and impute). The mean (0.066) is noticeably higher than the median (0.054), and the max (0.33) is far beyond the 75th percentile (0.095) — suggests a right-skewed distribution with potential outliers.
3. Item_MRP — No missing values. Mean (141.0) ≈ median (143.0), fairly symmetric distribution, std ~62. Range (31.3–266.9) seems plausible for product prices, no obvious outliers.
4. Outlet_Establishment_Year — Ranges 1985–2009, treated as numeric but is really ordinal/categorical. Useful to convert into "Outlet_Age" (e.g., current_year − establishment_year) as a feature for modeling.
5. Item_Outlet_Sales (target variable) — Mean (2181) > median (1794), and max (13,087) is ~4x the 75th percentile (3101) → strong right skew, likely with high-value outliers. A log-

transform may help if this is used as a regression target, and models should be evaluated with this skew in mind.

- Item_Visibility and Item_Outlet_Sales both show skew/outlier signals worth visualizing (histograms/boxplots) before modeling.
  - Categorical columns (Item_Type, Outlet_Type, Outlet_Size, etc.)

```
df_test.describe()
```

```
       Item_Weight  Item_Visibility     Item_MRP
Outlet_Establishment_Year
count  4705.000000      5681.000000  5681.000000
5681.000000
mean     12.695633         0.065684   141.023273
1997.828903
std       4.664849         0.051252    61.809091
8.372256
min       4.555000         0.000000    31.990000
1985.000000
25%       8.645000         0.027047    94.412000
1987.000000
50%      12.500000         0.054154   141.415400
1999.000000
75%      16.700000         0.093463   186.026600
2004.000000
max      21.350000         0.323637   266.588400
2009.000000
```

Merge data

```
data = pd.concat([df,df_test])
data.shape
```

```
(14204, 12)
```

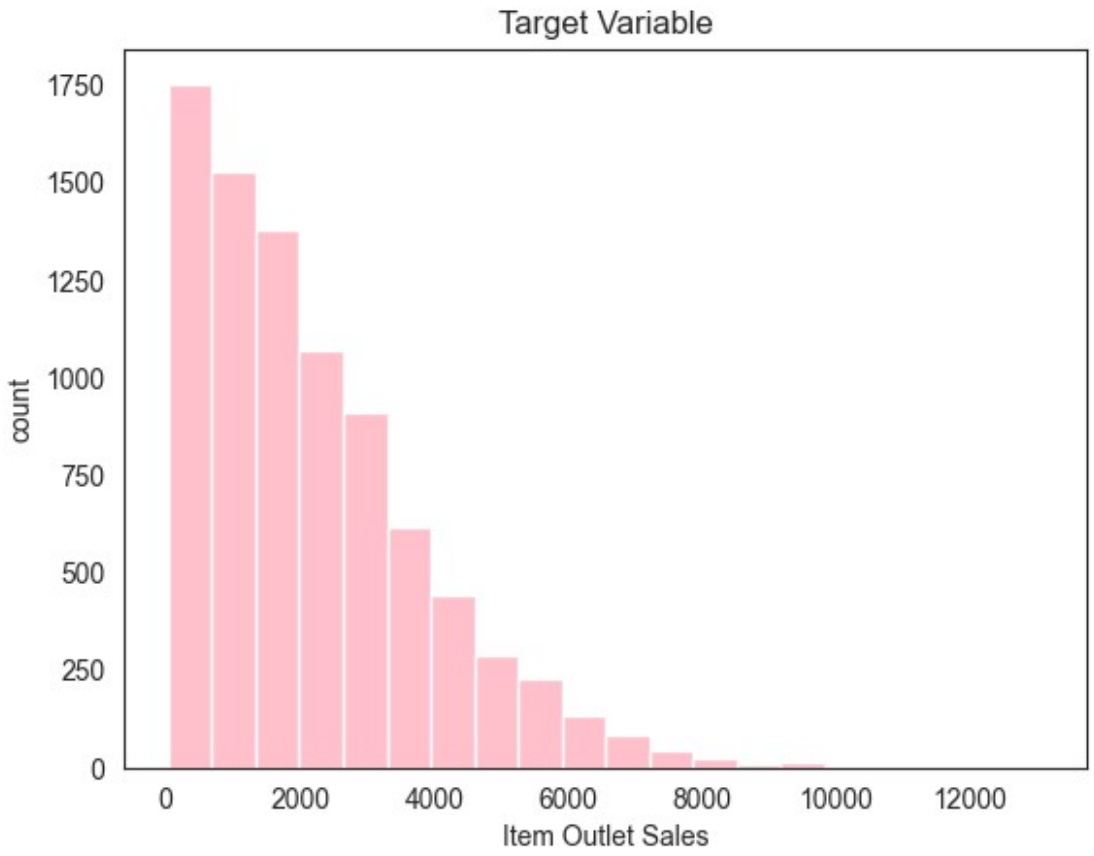
```
print(data['Item_Outlet_Sales'].isna().sum() == df_test.shape[0])
```

```
True
```

Observations:

It does not make sense to merge test as those do not have the Target variable. The only reason to use the merged data is to analyse the other variables

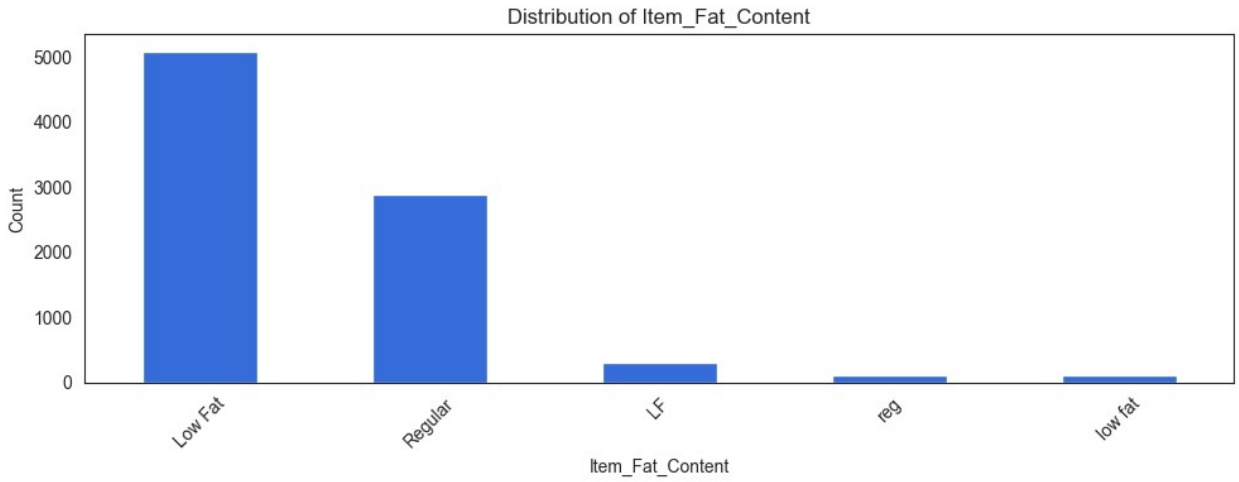
```
plt.hist(df['Item_Outlet_Sales'], bins = 20, color = 'pink')
plt.title('Target Variable')
plt.xlabel('Item Outlet Sales')
plt.ylabel('count')
plt.show()
```




```
def plot_column_distributions(df, columns=None, max_categories=20):
    """
    For each column, print value_counts and show a bar or hist plot.
    Columns with more than max_categories unique values get a
histogram;
    lower-cardinality columns get a bar chart.
    """
    if columns is None:
        columns = df.columns.tolist()

    for col in columns:
        print(f"\n--- {col} ---")
        print(df[col].value_counts())

print(df[col].value_counts(normalize=True).round(4).to_string())

        fig, ax = plt.subplots(figsize=(10, 4))
        vc = df[col].value_counts()

        if df[col].nunique() > max_categories:
            vc.plot.hist(ax=ax, bins=30)
        else:
            vc.plot.bar(ax=ax)
```

```
        ax.set_title(f'Distribution of {col}')
        ax.set_xlabel(col)
        ax.set_ylabel('Count')
        ax.tick_params(axis='x', rotation=45)
        plt.tight_layout()
        plt.show()

# Plot all columns in df — or pass a list to limit scope, e.g.:
plot_column_distributions(df, columns=['Item_Fat_Content'])
# plot_column_distributions(df)
```

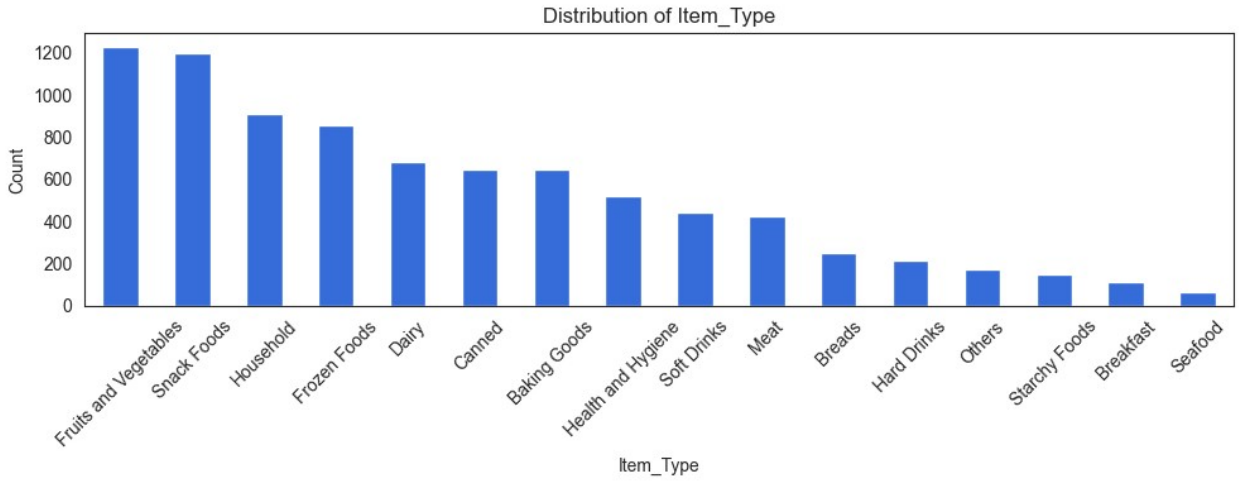
```
--- Item_Fat_Content ---
Item_Fat_Content
Low Fat    5089
Regular    2889
LF          316
reg         117
low fat     112
Name: count, dtype: int64
Item_Fat_Content
Low Fat    0.5971
Regular    0.3390
LF         0.0371
reg        0.0137
low fat    0.0131
```




```
plot_column_distributions(df, columns=['Item_Type'])
```

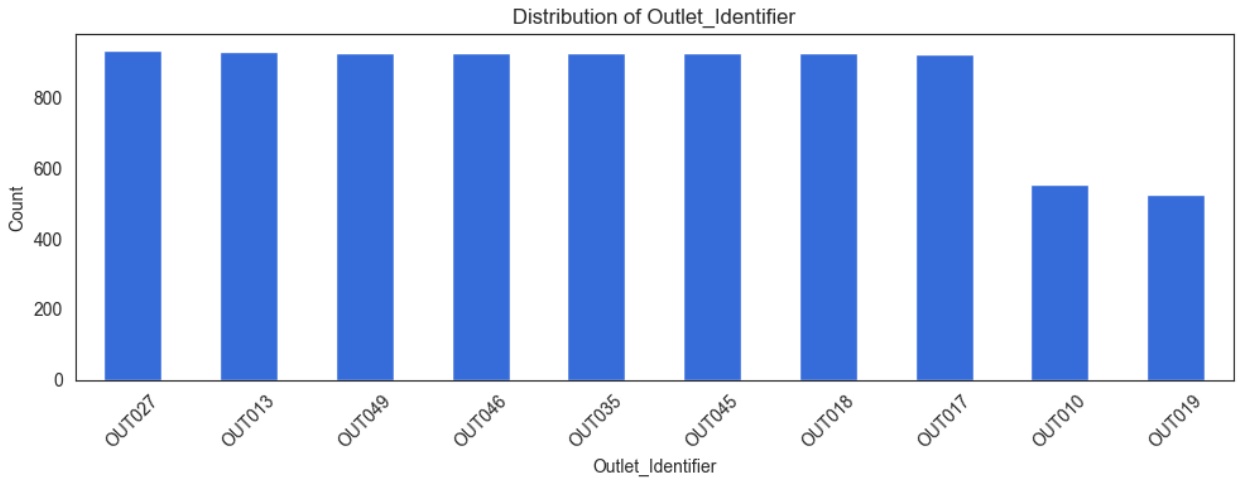
```
--- Item_Type ---
Item_Type
Fruits and Vegetables    1232
Snack Foods              1200
```

```
Household                910
Frozen Foods             856
Dairy                    682
Canned                   649
Baking Goods             648
Health and Hygiene       520
Soft Drinks              445
Meat                     425
Breads                   251
Hard Drinks              214
Others                   169
Starchy Foods            148
Breakfast                110
Seafood                   64
Name: count, dtype: int64
Item_Type
Fruits and Vegetables    0.1446
Snack Foods              0.1408
Household                0.1068
Frozen Foods             0.1004
Dairy                    0.0800
Canned                   0.0761
Baking Goods             0.0760
Health and Hygiene       0.0610
Soft Drinks              0.0522
Meat                     0.0499
Breads                   0.0294
Hard Drinks              0.0251
Others                   0.0198
Starchy Foods            0.0174
Breakfast                0.0129
Seafood                  0.0075
```



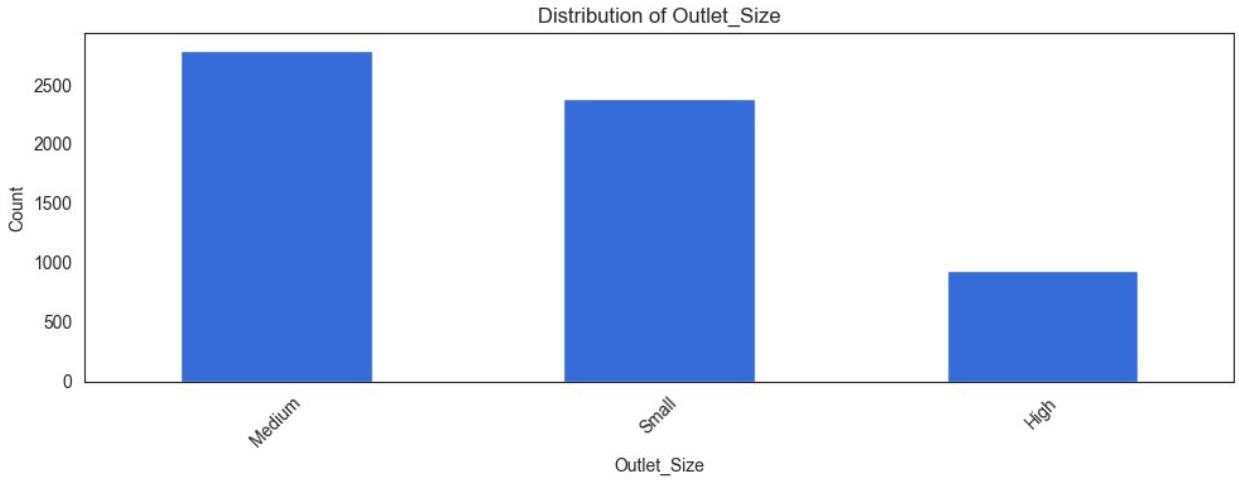

```
plot_column_distributions(df, columns=['Outlet_Identifier'])
```

```
--- Outlet_Identifier ---
Outlet_Identifier
OUT027    935
OUT013    932
OUT049    930
OUT046    930
OUT035    930
OUT045    929
OUT018    928
OUT017    926
OUT010    555
OUT019    528
Name: count, dtype: int64
Outlet_Identifier
OUT027    0.1097
OUT013    0.1094
OUT049    0.1091
OUT046    0.1091
OUT035    0.1091
OUT045    0.1090
OUT018    0.1089
OUT017    0.1086
OUT010    0.0651
OUT019    0.0620
```

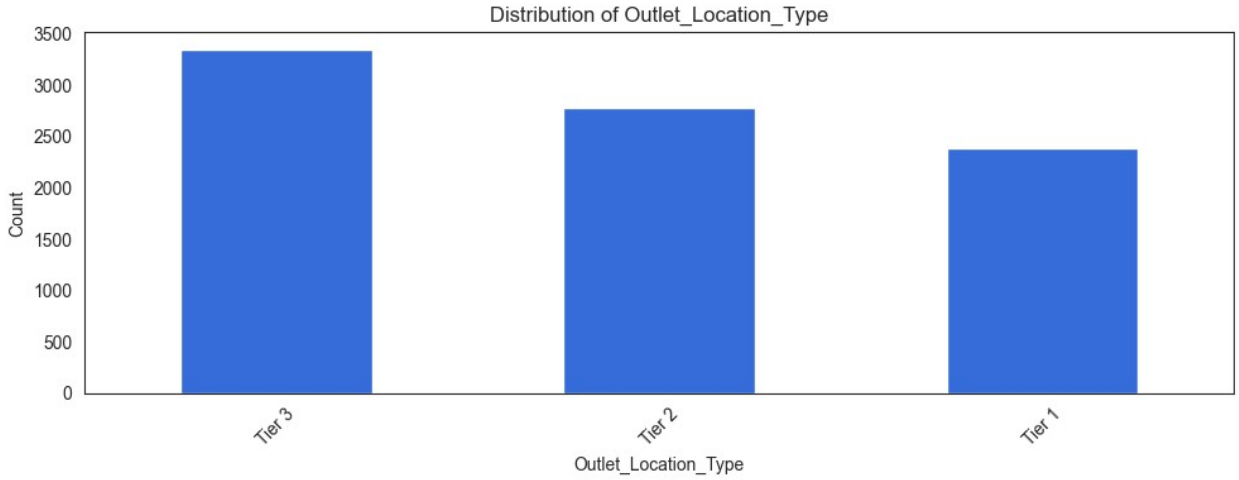



```
plot_column_distributions(df, columns=['Outlet_Size'])


--- Outlet_Size ---
Outlet_Size
Medium    2793
Small     2388
High       932
```

```
Name: count, dtype: int64
Outlet_Size
Medium    0.4569
Small     0.3906
High      0.1525
```

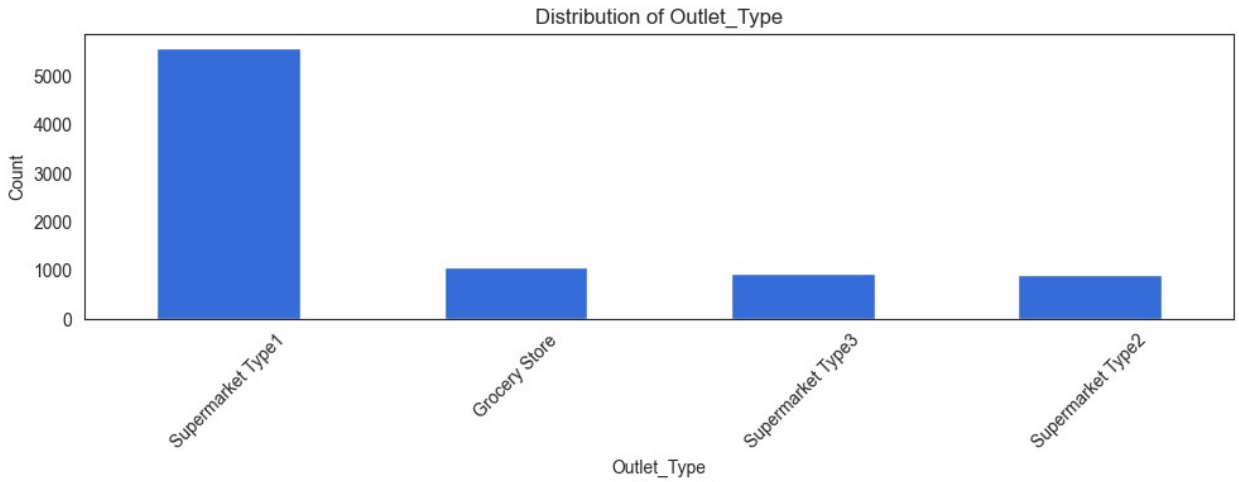



```
plot_column_distributions(df, columns=['Outlet_Location_Type'])


--- Outlet_Location_Type ---
Outlet_Location_Type
Tier 3    3350
Tier 2    2785
Tier 1    2388
Name: count, dtype: int64
Outlet_Location_Type
Tier 3    0.3931
Tier 2    0.3268
Tier 1    0.2802
```



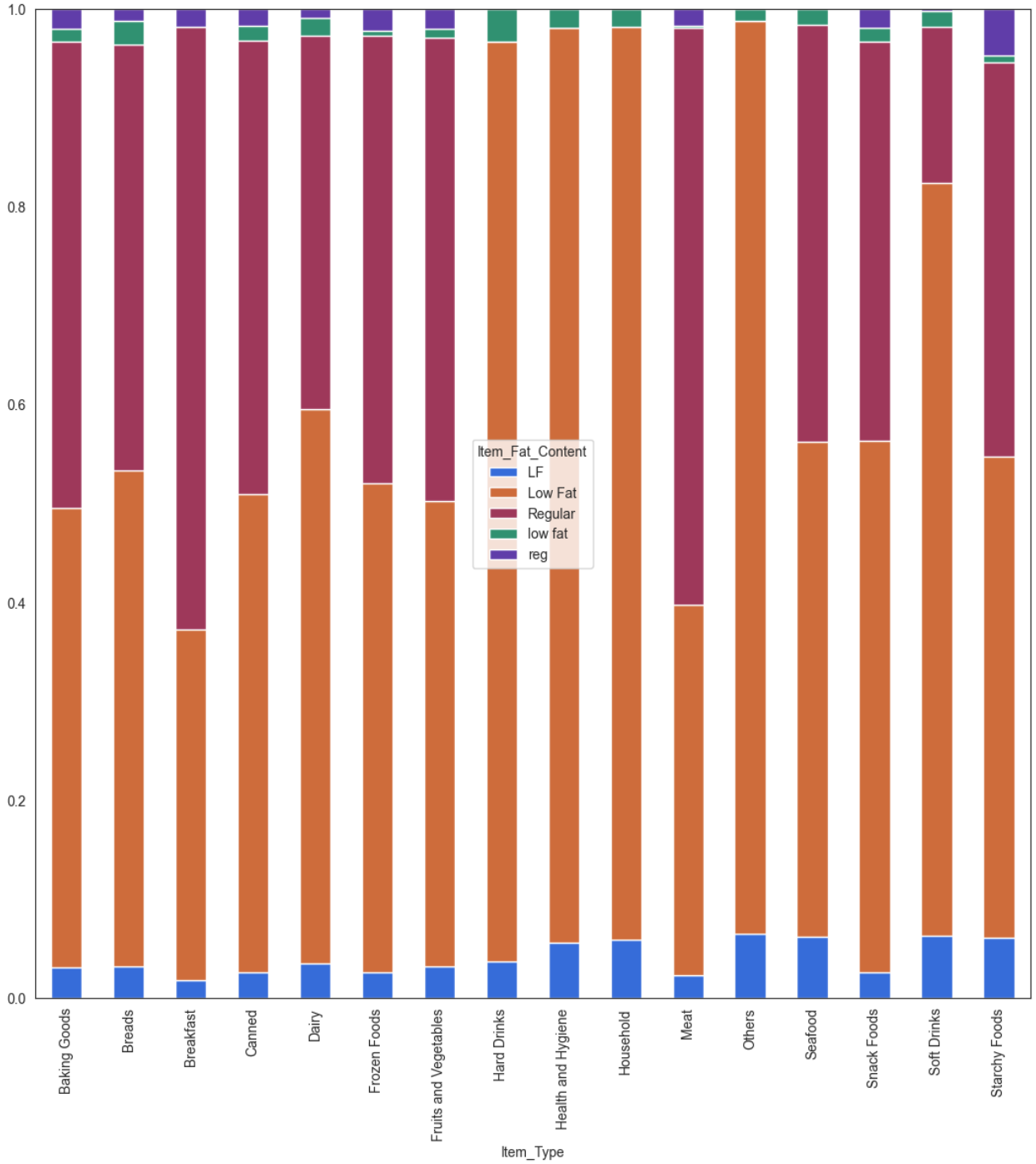
```
plot_column_distributions(df, columns=['Outlet_Type'])
```

```
--- Outlet_Type ---
Outlet_Type
Supermarket Type1    5577
Grocery Store        1083
Supermarket Type3     935
Supermarket Type2     928
Name: count, dtype: int64
Outlet_Type
Supermarket Type1    0.6543
Grocery Store        0.1271
Supermarket Type3    0.1097
Supermarket Type2    0.1089
```




```
Item_Type = pd.crosstab(df['Item_Type'], df['Item_Fat_Content'])
Item_Type.div(Item_Type.sum(1).astype(float), axis=0).plot(kind="bar",
stacked=True, figsize=(13, 13))
```

```
<Axes: xlabel='Item_Type'>
```




```
data.apply(lambda x : len(x.unique()))
```

```
Item_Identifier               1559
Item_Weight                    416
Item_Fat_Content                 5
Item_Visibility              13006
Item_Type                       16
Item_MRP                      8052
```

```
Outlet_Identifier               10
Outlet_Establishment_Year        9
Outlet_Size                      4
Outlet_Location_Type             3
Outlet_Type                      4
Item_Outlet_Sales             3494
dtype: int64
```

EDA - preprocess, before train-test split

Some of the data have data entry errors. For example, item_fat_content has value Regular and reg. This should be cleaned up across. Such imputations need not wait till split.

```
data['Item_Identifier'].apply(lambda x : x[0:2]).unique()
```

```
<ArrowStringArray>
['FD', 'DR', 'NC']
Length: 3, dtype: str
```

Observations:

['FD', 'DR', 'NC'] could map to Food, Drink and Non-Consumable

```
df['Item_Identifier_Type'] = df['Item_Identifier'].apply(lambda x:
x[0:2])
df_test['Item_Identifier_Type'] =
df_test['Item_Identifier'].apply(lambda x: x[0:2])
df['Item_Identifier_Type'].unique()
```

```
<ArrowStringArray>
['FD', 'DR', 'NC']
Length: 3, dtype: str
```

Since Non Consumables cannot be low fat or regular so will create new var in item fat content for non consumable item identifier

```
df.loc[df['Item_Identifier_Type']=='NC','Item_Fat_Content']='No Fat'
df_test.loc[df_test['Item_Identifier_Type']=='NC','Item_Fat_Content']=
'No Fat'

df.head()
```

```
  Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
0           FDA15         9.30          Low Fat         0.016047
1           DRC01         5.92          Regular         0.019278
2           FDN15        17.50          Low Fat         0.016760
3           FDX07        19.20          Regular         0.000000
4           NCD19         8.93           No Fat         0.000000
```

```
               Item_Type  Item_MRP Outlet_Identifier  \
0                  Dairy  249.8092            OUT049
1            Soft Drinks   48.2692            OUT018
2                   Meat  141.6180            OUT049
3  Fruits and Vegetables  182.0950            OUT010
4              Household   53.8614            OUT013

   Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  \
0                       1999      Medium               Tier 1
1                       2009      Medium               Tier 3
2                       1999      Medium               Tier 1
3                       1998         NaN               Tier 3
4                       1987        High               Tier 3

         Outlet_Type  Item_Outlet_Sales Item_Identifier_Type
0  Supermarket Type1          3735.1380                   FD
1  Supermarket Type2           443.4228                   DR
2  Supermarket Type1          2097.2700                   FD
3      Grocery Store           732.3800                   FD
4  Supermarket Type1           994.7052                   NC
```

```
cat_features =
df.select_dtypes(include=['object','category']).columns.tolist()
cat_features
```

```
['Item_Identifier',
 'Item_Fat_Content',
 'Item_Type',
 'Outlet_Identifier',
 'Outlet_Size',
 'Outlet_Location_Type',
 'Outlet_Type',
 'Item_Identifier_Type']
```

```
df['Item_Fat_Content'].value_counts()
```

```
Item_Fat_Content
Low Fat    3612
Regular    2889
No Fat     1599
LF          222
reg         117
low fat      84
Name: count, dtype: int64
```

we can see that that there is mistake in collection the data for Item_Fat_Content .

For Low Fat they have used three symbols which are Low Fat, LF an low fat so we will replace that values by original values that areLow fat and regular

```
df['Item_Fat_Content'] = df['Item_Fat_Content'].replace('LF','Low
Fat')
df['Item_Fat_Content'] = df['Item_Fat_Content'].replace('low fat','Low
Fat')
df['Item_Fat_Content'] =
df['Item_Fat_Content'].replace('reg','Regular')

df_test['Item_Fat_Content'] =
df_test['Item_Fat_Content'].replace('LF','Low Fat')
df_test['Item_Fat_Content'] = df_test['Item_Fat_Content'].replace('low
fat','Low Fat')
df_test['Item_Fat_Content'] =
df_test['Item_Fat_Content'].replace('reg','Regular')
```

Filling the missing values

```
df.isnull().sum()
```

```
Item_Identifier                 0
Item_Weight                  1463
Item_Fat_Content                0
Item_Visibility                 0
Item_Type                       0
Item_MRP                        0
Outlet_Identifier               0
Outlet_Establishment_Year       0
Outlet_Size                  2410
Outlet_Location_Type            0
Outlet_Type                     0
Item_Outlet_Sales               0
Item_Identifier_Type            0
dtype: int64
```

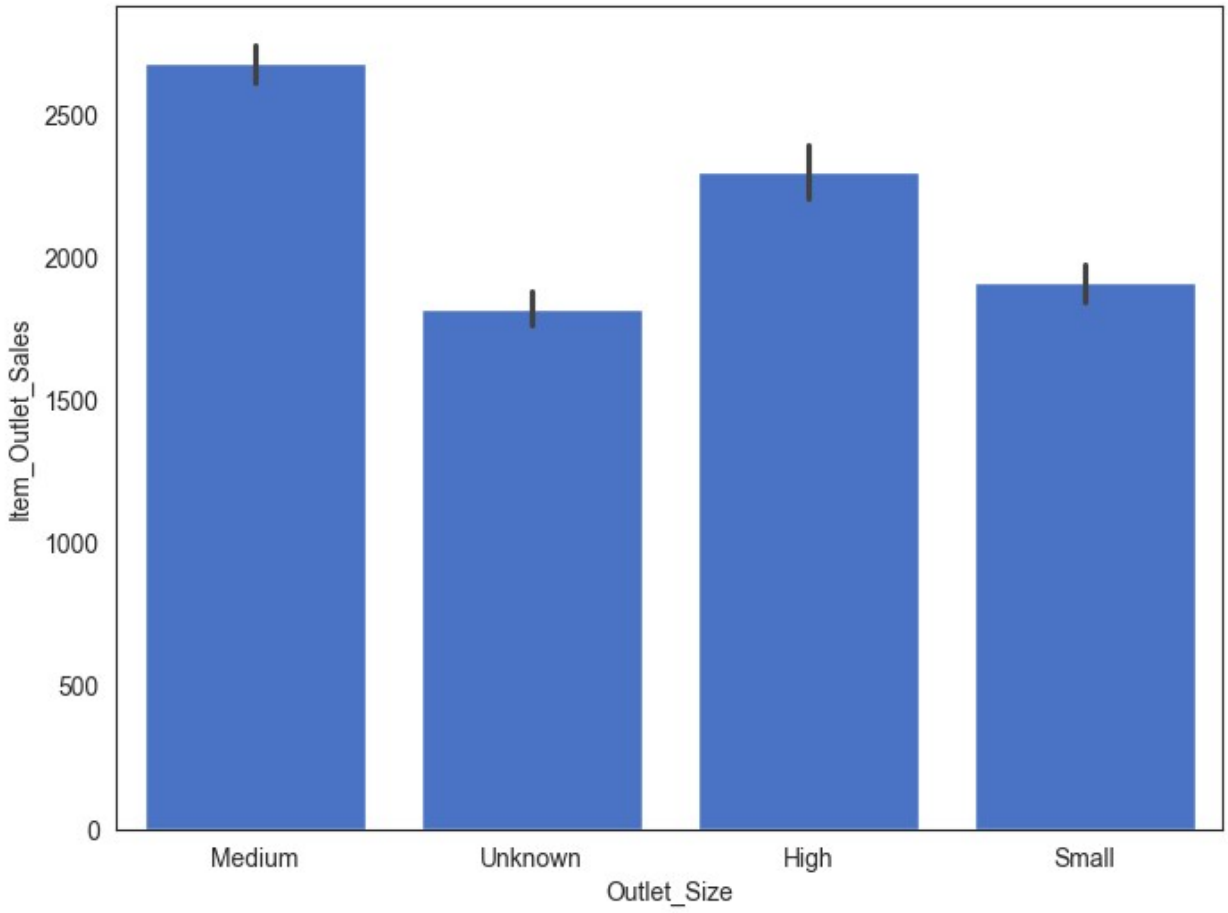
```
#An item in the shelf cannot have Item Visibility of 0
df['Item_Visibility'].value_counts()
```

```
Item_Visibility
0.000000    526
0.076975      3
0.026818      2
0.071958      2
0.074613      2
           ...
0.056783      1
0.046982      1
0.035186      1
0.145221      1
0.044878      1
Name: count, Length: 7880, dtype: int64
```

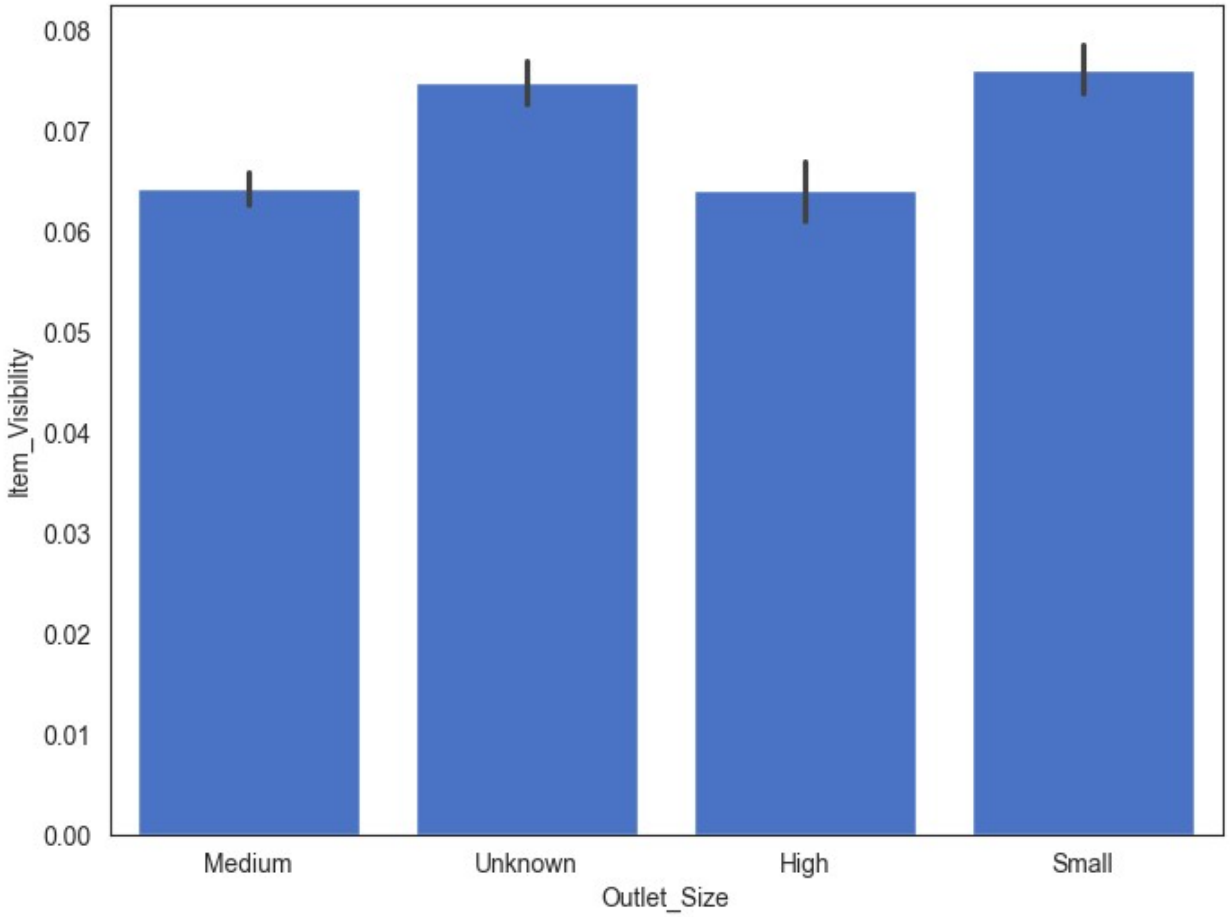
```
df['Item_Visibility'] = df['Item_Visibility'].replace(0,np.nan)
df_test['Item_Visibility'] =
df_test['Item_Visibility'].replace(0,np.nan)

df['Outlet_Size'].isna().sum()

np.int64(2410)
```

Outlet size cannot be na

```
df['Outlet_Size'] = df['Outlet_Size'].fillna('Unknown')
df_test['Outlet_Size'] = df_test['Outlet_Size'].fillna('Unknown')

# plt.figure(figsize=(8,6))
# sns.barplot(df['Outlet_Size'],df['Item_Outlet_Sales'])
# plt.show()

plt.figure(figsize=(8,6))
sns.barplot(x='Outlet_Size', y='Item_Outlet_Sales', data=df)
plt.show()
```



```
plt.figure(figsize=(8,6))
sns.barplot(x='Outlet_Size', y='Item_Visibility', data=df)
plt.show()
```

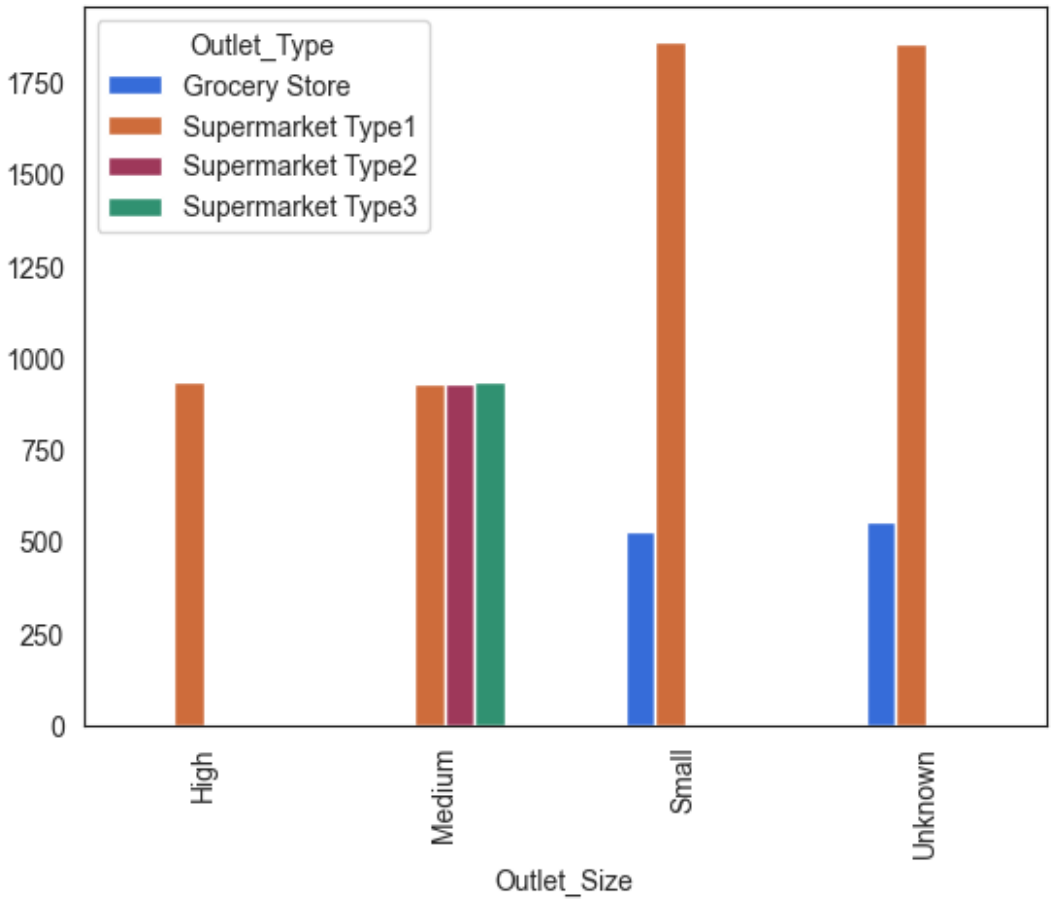


From above two graph we can conclude as follows:

Graph 1 : Generally there is low outlet sales in stores having outlet size small and unknown . Also store having small outlet size resembles the store having unknown outlet size

Graph 1 : Generally there is high visibility in stores having outlet size small and unknown . Also store having small outlet size resembles the store having unknown outlet size

```
a=pd.crosstab(df['Outlet_Size'],df['Outlet_Type'])
a.plot(kind='bar')
plt.show()
```



Above graph also tells us that outlet having small size resembles the outlet having unknown size.Also outlet having smaller size are supermaket type 1 or grocery store

```
# df1=df[df['Outlet_Size'] == 'Unknown']
# df1.head()
df[df['Outlet_Size'] == 'Unknown']
```

```
     Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
3              FDX07       19.200          Regular              NaN
8              FDH17       16.200          Regular         0.016687
9              FDU28       19.200          Regular         0.094450
25             NCD06       13.000           No Fat         0.099887
28             FDE51        5.925          Regular         0.161467
...              ...          ...              ...              ...
8502           NCH43        8.420           No Fat         0.070712
8508           FDW31       11.350          Regular         0.043246
8509           FDG45        8.100          Low Fat         0.214306
8514           FDA01       15.000          Regular         0.054489
8519           FDS36        8.380          Regular         0.046982

                  Item_Type  Item_MRP Outlet_Identifier  \
3     Fruits and Vegetables  182.0950            OUT010
```

```
8              Frozen Foods   96.9726            OUT045
9              Frozen Foods  187.8214            OUT017
25                Household   45.9060            OUT017
28                    Dairy   45.5086            OUT010
...                     ...       ...               ...
8502              Household  216.4192            OUT045
8508  Fruits and Vegetables  199.4742            OUT045
8509  Fruits and Vegetables  213.9902            OUT010
8514                 Canned   57.5904            OUT045
8519           Baking Goods  108.1570            OUT045

      Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  \
3                          1998     Unknown               Tier 3
8                          2002     Unknown               Tier 2
9                          2007     Unknown               Tier 2
25                         2007     Unknown               Tier 2
28                         1998     Unknown               Tier 3
...                         ...         ...                  ...
8502                       2002     Unknown               Tier 2
8508                       2002     Unknown               Tier 2
8509                       1998     Unknown               Tier 3
8514                       2002     Unknown               Tier 2
8519                       2002     Unknown               Tier 2

            Outlet_Type  Item_Outlet_Sales Item_Identifier_Type
3         Grocery Store           732.3800                  FD
8     Supermarket Type1          1076.5986                  FD
9     Supermarket Type1          4710.5350                  FD
25    Supermarket Type1           838.9080                  NC
28        Grocery Store           178.4344                  FD
...                 ...                ...                 ...
8502  Supermarket Type1          3020.0688                  NC
8508  Supermarket Type1          2587.9646                  FD
8509      Grocery Store           424.7804                  FD
8514  Supermarket Type1           468.7232                  FD
8519  Supermarket Type1           549.2850                  FD

[2410 rows x 13 columns]
```

```
# df1['Outlet_Type'].unique()
```

This dataframe also supports the previous reason that outlet having unknown size are of type grocery store or supermarket type 1

So i'll fill the missing values of outlet size by small as there are enough evidence

```
df['Outlet_Size'] = df['Outlet_Size'].replace('Unknown','Small')
df_test['Outlet_Size'] =
df_test['Outlet_Size'].replace('Unknown','Small')
```

```
target = 'Item_Outlet_Sales'

traincols = set(df.columns)
testcols=set(df_test.columns)

assert (

    set(testcols).issubset(traincols)
    and len(df.columns) - len(df_test.columns) == 1
    and (traincols - testcols) == {target}
)

df.isnull().sum()
```

```
Item_Identifier                 0
Item_Weight                  1463
Item_Fat_Content                0
Item_Visibility               526
Item_Type                       0
Item_MRP                        0
Outlet_Identifier               0
Outlet_Establishment_Year       0
Outlet_Size                     0
Outlet_Location_Type            0
Outlet_Type                     0
Item_Outlet_Sales               0
Item_Identifier_Type            0
dtype: int64
```

```
seed=42
global_test_size=.2

from sklearn.model_selection import train_test_split

X= df.drop(columns=[target])
y=df[target]

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=global_test_size,random_state=seed)

X_train_copy = X_train.copy()
X_test_copy = X_test.copy()
```

Now Impute missing features using X_Train

```
df['Item_Visibility'].isna().sum()
```

```
np.int64(526)
```

```
df[df['Item_Visibility'].isna()]
```

```
     Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
3              FDX07       19.200          Regular              NaN
4              NCD19        8.930          No Fat              NaN
5              FDP36       10.395          Regular              NaN
10             FDY07       11.800          Low Fat              NaN
32             FDP33       18.700          Low Fat              NaN
...              ...          ...              ...              ...
8480           FDQ58          NaN          Low Fat              NaN
8484           DRJ49        6.865          Low Fat              NaN
8486           FDR20       20.000          Regular              NaN
8494           NCI54       15.200          No Fat              NaN
8500           NCQ42       20.350          No Fat              NaN

                  Item_Type  Item_MRP Outlet_Identifier  \
3     Fruits and Vegetables  182.0950            OUT010
4                 Household   53.8614            OUT013
5              Baking Goods   51.4008            OUT018
10    Fruits and Vegetables   45.5402            OUT049
32              Snack Foods  256.6672            OUT018
...                     ...      ...               ...
8480            Snack Foods  154.5340            OUT019
8484            Soft Drinks  129.9652            OUT013
8486  Fruits and Vegetables   46.4744            OUT010
8494              Household  110.4912            OUT017
8500              Household  125.1678            OUT017

      Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  \
3                          1998       Small               Tier 3
4                          1987        High               Tier 3
5                          2009      Medium               Tier 3
10                         1999      Medium               Tier 1
32                         2009      Medium               Tier 3
...                         ...         ...                  ...
8480                       1985       Small               Tier 1
8484                       1987        High               Tier 3
8486                       1998       Small               Tier 3
8494                       2007       Small               Tier 2
8500                       2007       Small               Tier 2

            Outlet_Type  Item_Outlet_Sales Item_Identifier_Type
3         Grocery Store           732.3800                  FD
4     Supermarket Type1           994.7052                  NC
5     Supermarket Type2           556.6088                  FD
10    Supermarket Type1          1516.0266                  FD
32    Supermarket Type2          3068.0064                  FD
...                 ...                ...                 ...
8480      Grocery Store           459.4020                  FD
8484  Supermarket Type1          2324.9736                  DR
8486      Grocery Store            45.2744                  FD
8494  Supermarket Type1          1637.8680                  NC
```

```
8500  Supermarket Type1          1907.5170                    NC

[526 rows x 13 columns]
```

Why Item_Weight is a Good Choice

Highest Correlation with Missingness:

From your earlier analysis, Item_Weight had the highest percentage of NaN values (~17.11%) in the rows where Item_Visibility is NaN. This suggests that Item_Weight and Item_Visibility might be related (e.g., heavier items may have different visibility patterns).

Domain Logic:

In retail, item weight can influence shelf visibility (e.g., heavier items might be placed differently). Grouping by Item_Weight (or Item_Identifier + Item_Weight) can capture item-specific visibility trends.

```
df['Item_Weight'].isnull().sum()
```

```
np.int64(1463)
```

Use a hierarchy of grouping to handle cases where Item_Weight is missing:

Impute Item_Weight and Visibility

```
df[df['Item_Weight'].isna()]
```

```
     Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
7              FDP10          NaN          Low Fat         0.127470
18             DRI11          NaN          Low Fat         0.034238
21             FDW12          NaN          Regular         0.035400
23             FDC37          NaN          Low Fat         0.057557
29             FDC14          NaN          Regular         0.072222
...              ...          ...              ...              ...
8485           DRK37          NaN          Low Fat         0.043792
8487           DRG13          NaN          Low Fat         0.037006
8488           NCN14          NaN           No Fat         0.091473
8490           FDU44          NaN          Regular         0.102296
8504           NCN18          NaN           No Fat         0.124111

                  Item_Type  Item_MRP Outlet_Identifier  \
7               Snack Foods  107.7622            OUT027
18              Hard Drinks  113.2834            OUT027
21             Baking Goods  144.5444            OUT027
23             Baking Goods  107.6938            OUT019
29                   Canned   43.6454            OUT019
...                     ...       ...               ...
8485            Soft Drinks  189.0530            OUT027
8487            Soft Drinks  164.7526            OUT027
8488                 Others  184.6608            OUT027
```

```
8490  Fruits and Vegetables  162.3552            OUT019
8504              Household  111.7544            OUT027

      Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  \
7                          1985      Medium               Tier 3
18                         1985      Medium               Tier 3
21                         1985      Medium               Tier 3
23                         1985       Small               Tier 1
29                         1985       Small               Tier 1
...                         ...         ...                  ...
8485                       1985      Medium               Tier 3
8487                       1985      Medium               Tier 3
8488                       1985      Medium               Tier 3
8490                       1985       Small               Tier 1
8504                       1985      Medium               Tier 3

            Outlet_Type  Item_Outlet_Sales Item_Identifier_Type
7     Supermarket Type3          4022.7636                   FD
18    Supermarket Type3          2303.6680                   DR
21    Supermarket Type3          4064.0432                   FD
23        Grocery Store           214.3876                   FD
29        Grocery Store           125.8362                   FD
...                 ...                ...                  ...
8485  Supermarket Type3          6261.8490                   DR
8487  Supermarket Type3          4111.3150                   DR
8488  Supermarket Type3          2756.4120                   NC
8490      Grocery Store           487.3656                   FD
8504  Supermarket Type3          4138.6128                   NC

[1463 rows x 13 columns]
```

```
vis_mean_by_id = X_train.groupby('Item_Identifier')
['Item_Visibility'].mean()
vis_global_mean = X_train['Item_Visibility'].mean()  # The absolute
safety net
for d in (X_train, X_test, df_test):
    d['Item_Visibility'] =
d['Item_Visibility'].fillna(d['Item_Identifier'].map(vis_mean_by_id))
    # Step 2: Catch any remaining NaNs with the global training mean
    d['Item_Visibility'] =
d['Item_Visibility'].fillna(vis_global_mean)

item_weight_mean_by_id = X_train.groupby('Item_Identifier')
['Item_Weight'].mean()
item_weight_median_by_type = X_train.groupby('Item_Type')
['Item_Weight'].median()
for d in (X_train, X_test, df_test):
    d['Item_Weight'] =
d['Item_Weight'].fillna(d['Item_Identifier'].map(item_weight_mean_by_i
d))
```

```
    d['Item_Weight'] =
d['Item_Weight'].fillna(d['Item_Type'].map(item_weight_median_by_type)
)

X_train['Item_Weight'].isna().sum()
```

```
np.int64(0)
```

```
assert X_train['Item_Visibility'].isna().sum() ==
X_train['Item_Visibility'].isnull().sum()
```

Preprocessing is completed. This should be a good placeholder for saving files for pyod notebook

```
precleaned = pd.concat([
    X_train.assign(**{target: y_train}),
    X_test.assign(**{target: y_test}),
    ]).sort_index()
precleaned.to_csv('data/train_precleaned.csv',index=False)
print(f"Saved precleaned data to data/train_precleaned.csv")
```

```
Saved precleaned data to data/train_precleaned.csv
```

```
df_test.to_csv('data/test_precleaned.csv',index=False)
print(f"Saved precleaned data to data/test_precleaned.csv")
```

```
Saved precleaned data to data/test_precleaned.csv
```

Outlier handling — fit cap threshold on X_train only; X_test/df_test untouched

```
## Several techiniques like z-score, iqr or using std deviation

q1, q3 = X_train['Item_Visibility'].quantile([.25,.75])
iqr = q3 -q1
upper_bound = q3 + 1.5 * iqr
replacement = X_train['Item_Visibility'].mean()

for d in (X_train,X_test,df_test):
    d['Item_Visibility'] = d['Item_Visibility'].apply(lambda  x :
replacement if x > upper_bound else x)

X_Train_1 =X_train.copy()
X_test1= X_test.copy()
df_test1=df_test.copy()

# X_train=X_Train_1.copy()
# X_test=X_test1.copy()
# df_test=df_test1.copy()
```

```
def get_datasets():
    return (X_train,X_test,df_test)
```

Additional New Features post split

```
mrp_bin_labels = ['cheap', 'affordable', 'slightly expensive',
'expensive']
weight_bin_labels = ['vlight', 'light', 'moderate', 'heavy']
X_train['MRP_bin'], mrp_bins = pd.qcut(X_train['Item_MRP'], q=4,
labels=mrp_bin_labels, retbins=True)
X_train['Weight_bin'], weight_bins = pd.qcut(X_train['Item_Weight'],
q=4, labels=weight_bin_labels, retbins=True)
mrp_bins[0], mrp_bins[-1] = -np.inf, np.inf
weight_bins[0], weight_bins[-1] = -np.inf, np.inf

for d in (X_test, df_test):
    d['MRP_bin'] = pd.cut(d['Item_MRP'], bins=mrp_bins,
labels=mrp_bin_labels)
    d['Weight_bin'] = pd.cut(d['Item_Weight'], bins=weight_bins,
labels=weight_bin_labels)

calorie_map = {
    'Dairy': 46,
    'Soft Drinks': 51,
    'Meat': 143,
    'Fruits and Vegetables': 65,
    'Baking Goods': 140,
    'Snack Foods': 475,
    'Frozen Foods': 50,
    'Breakfast': 350,
    'Hard Drinks': 250,
    'Canned': 80,
    'Starchy Foods': 90,
    'Seafood': 204,
    'Breads': 250,
}

item_frequency_map={'Dairy':'D','Meat':'S','Fruits and
Vegetables':'D','Breakfast':'S','Breads':'S','Starchy
Foods':'S','Seafood':'S','Soft Drinks':'S','Household':'D','Baking
Goods':'S','Snack Foods':'D','Frozen Foods':'D','Hard
Drinks':'S','Canned':'D','Health and Hygiene':'S','Others':'S'}

    # calorie_map / item_frequency_map are fixed domain dicts (not fit
from data) -- apply to all three directly
for d in (X_train, X_test, df_test):
    d['Calorie_Count_per_100g'] =
d['Item_Type'].map(calorie_map).fillna(0)
    d['Calorie_Count_per_givenwt'] = d['Calorie_Count_per_100g'] / 100
* d['Item_Weight']
```

```
    d['Item_Frequency'] = d['Item_Type'].map(item_frequency_map)
    d['Outlet_Existence'] = 2020 - d['Outlet_Establishment_Year']
    d['Outlet_Status'] = (d['Outlet_Establishment_Year'] >
2000).astype(int)

X_train.columns
```

```
Index(['Item_Identifier', 'Item_Weight', 'Item_Fat_Content',
'Item_Visibility',
       'Item_Type', 'Item_MRP', 'Outlet_Identifier',
       'Outlet_Establishment_Year', 'Outlet_Size',
'Outlet_Location_Type',
       'Outlet_Type', 'Item_Identifier_Type', 'MRP_bin', 'Weight_bin',
       'Calorie_Count_per_100g', 'Calorie_Count_per_givenwt',
'Item_Frequency',
       'Outlet_Existence', 'Outlet_Status'],
      dtype='str')
```

Skewness

Diagnose on X_train only

```
print(

X_train[['Item_Weight','Item_MRP','Item_Visibility','Calorie_Count_per
_givenwt']].skew()
)
```

```
Item_Weight                 0.069549
Item_MRP                    0.106298
Item_Visibility             0.768092
Calorie_Count_per_givenwt    1.992158
dtype: float64
```

```
## Lets apply for skew > 1

# log/sqrt are parameter-free, so safe to apply identically everywhere
once the check says they're needed
for d in get_datasets():
    d['log_visibility'] = np.log(d['Item_Visibility'])
    d['log_Calorie'] = np.sqrt(d['Calorie_Count_per_givenwt'])

X_train.head()
```

```
     Item_Identifier  Item_Weight Item_Fat_Content  Item_Visibility  \
549             FDW44        9.500          Regular         0.035206
7757            NCF54       18.000           No Fat         0.047473
764             FDY03       17.600          Regular         0.076122
6867            FDQ20        8.325          Low Fat         0.029845
2716            FDP34       12.850          Low Fat         0.137228
```

```
                  Item_Type  Item_MRP Outlet_Identifier  \
549   Fruits and Vegetables  171.3448            OUT049
7757              Household  170.5422            OUT045
764                    Meat  111.7202            OUT046
6867  Fruits and Vegetables   41.6138            OUT045
2716            Snack Foods  155.5630            OUT046

      Outlet_Establishment_Year Outlet_Size Outlet_Location_Type  ...
\
549                        1999      Medium               Tier 1  ...

7757                       2002       Small               Tier 2  ...

764                        1997       Small               Tier 1  ...

6867                       2002       Small               Tier 2  ...

2716                       1997       Small               Tier 1  ...


     Item_Identifier_Type             MRP_bin Weight_bin  \
549                    FD  slightly expensive      light
7757                   NC  slightly expensive      heavy
764                    FD          affordable      heavy
6867                   FD              cheap     vlight
2716                   FD  slightly expensive   moderate

     Calorie_Count_per_100g  Calorie_Count_per_givenwt  Item_Frequency
\
549                    65.0                    6.17500               D

7757                    0.0                    0.00000               D

764                   143.0                   25.16800               S

6867                   65.0                    5.41125               D

2716                  475.0                   61.03750               D


     Outlet_Existence  Outlet_Status  log_visibility  log_Calorie
549                21              0       -3.346543     2.484955
7757               18              1       -3.047591     0.000000
764                23              0       -2.575420     5.016772
6867               18              1       -3.511730     2.326209
2716               23              0       -1.986113     7.812650

[5 rows x 21 columns]
```

Label Encoding

```
from sklearn.preprocessing import LabelEncoder

label_encoders = {}

#Label Encoding Types
for col in ['Item_Identifier_Type', 'Item_Type', 'Outlet_Type',
'Outlet_Identifier']:
    le= LabelEncoder()
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])
    df_test[col] = le.transform(df_test[col])
    label_encoders[col] = le

#Ordinal Map Encoding types
ordinal_maps = {
    'Outlet_Size': {'Small': 0, 'Medium': 1, 'High': 2},
    'MRP_bin': {'cheap': 0, 'affordable': 1, 'slightly expensive': 2,
'expensive': 3},
    'Weight_bin': {'vlight': 0, 'light': 1, 'moderate': 2, 'heavy':
3},
    'Outlet_Location_Type': {'Tier 3': 0, 'Tier 2': 1, 'Tier 1': 2},
    'Item_Fat_Content': {'No Fat': 0, 'Low Fat': 1, 'Regular': 2},
    'Item_Frequency': {'D': 1, 'S': 0},
}

for col, mapping in ordinal_maps.items():
    for d in get_datasets():
        d[col] = d[col].map(mapping)

#Rank-encoding for features with years
# Outlet_Existence and Outlet_Establishment_Year are rank-encodings of
the same years
# -> fit the rank on X_train's years only, reuse on X_test/df_test
year_rank = {year: rank for rank, year in
enumerate(sorted(X_train['Outlet_Establishment_Year'].unique()))}
existence_rank = {2020 - year: rank for year, rank in
year_rank.items()}

for d in get_datasets():
    d['Outlet_Existence'] = d['Outlet_Existence'].map(existence_rank)
    d['Outlet_Establishment_Year'] =
d['Outlet_Establishment_Year'].map(year_rank)

X_train.head()
```

```
     Item_Identifier  Item_Weight  Item_Fat_Content
Item_Visibility  \
549            FDW44        9.500                 2         0.035206
```

```
7757           NCF54       18.000                  0         0.047473

764            FDY03       17.600                  2         0.076122

6867           FDQ20        8.325                  1         0.029845

2716           FDP34       12.850                  1         0.137228


      Item_Type  Item_MRP  Outlet_Identifier
Outlet_Establishment_Year  \
549           6  171.3448                  9
4
7757          9  170.5422                  7
5
764          10  111.7202                  8
2
6867          6   41.6138                  7
5
2716         13  155.5630                  8
2

      Outlet_Size  Outlet_Location_Type  ...  Item_Identifier_Type
MRP_bin  \
549             1                     2  ...                     1
2
7757            0                     1  ...                     2
2
764             0                     2  ...                     1
1
6867            0                     1  ...                     1
0
2716            0                     2  ...                     1
2

     Weight_bin Calorie_Count_per_100g  Calorie_Count_per_givenwt  \
549          1                   65.0                    6.17500
7757         3                    0.0                    0.00000
764          3                  143.0                   25.16800
6867         0                   65.0                    5.41125
2716         2                  475.0                   61.03750

      Item_Frequency  Outlet_Existence  Outlet_Status  log_visibility
\
549                1                 4              0       -3.346543

7757               1                 5              1       -3.047591

764                0                 2              0       -2.575420
```

```
6867                1                   5                1       -3.511730

2716                1                   2                0       -1.986113


      log_Calorie
549      2.484955
7757     0.000000
764      5.016772
6867     2.326209
2716     7.812650

[5 rows x 21 columns]
```

Scale — fit StandardScaler on X_train only

```
from sklearn.preprocessing import StandardScaler

scale_features = ['Item_Weight', 'Item_MRP', 'log_visibility',
'log_Calorie']
# raw skewed duplicates dropped per the earlier skew check
scaler = StandardScaler()
X_train[scale_features] =
scaler.fit_transform(X_train[scale_features])
X_test[scale_features] = scaler.transform(X_test[scale_features])
df_test[scale_features] = scaler.transform(df_test[scale_features])

X_train.head()
```

```
     Item_Identifier  Item_Weight  Item_Fat_Content
Item_Visibility  \
549             FDW44    -0.736531                 2         0.035206

7757            NCF54     1.096585                 0         0.047473

764             FDY03     1.010321                 2         0.076122

6867            FDQ20    -0.989933                 1         0.029845

2716            FDP34    -0.014068                 1         0.137228


      Item_Type  Item_MRP  Outlet_Identifier
Outlet_Establishment_Year  \
549           6  0.470709                  9
4
7757          9  0.457877                  7
5
764          10 -0.482625                  8
```

```
2
6867           6 -1.603553                    7
5
2716          13  0.218375                    8
2

      Outlet_Size  Outlet_Location_Type  ...  Item_Identifier_Type
MRP_bin  \
549             1                     2  ...                     1
2
7757            0                     1  ...                     2
2
764             0                     2  ...                     1
1
6867            0                     1  ...                     1
0
2716            0                     2  ...                     1
2

     Weight_bin Calorie_Count_per_100g  Calorie_Count_per_givenwt  \
549           1                  65.0                    6.17500
7757          3                   0.0                    0.00000
764           3                 143.0                   25.16800
6867          0                  65.0                    5.41125
2716          2                 475.0                   61.03750

      Item_Frequency  Outlet_Existence  Outlet_Status  log_visibility
\
549                1                 4              0       -0.532978

7757               1                 5              1       -0.141163

764                0                 2              0        0.477679

6867               1                 5              1       -0.749477

2716               1                 2              0        1.250044


      log_Calorie
549     -0.331257
7757    -1.338259
764      0.694736
6867    -0.395587
2716     1.827737

[5 rows x 21 columns]

X_train.info()
```

```
<class 'pandas.DataFrame'>
Index: 6818 entries, 549 to 7270
Data columns (total 21 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Item_Identifier             6818 non-null   str
 1   Item_Weight                 6818 non-null   float64
 2   Item_Fat_Content            6818 non-null   int64
 3   Item_Visibility             6818 non-null   float64
 4   Item_Type                   6818 non-null   int64
 5   Item_MRP                    6818 non-null   float64
 6   Outlet_Identifier           6818 non-null   int64
 7   Outlet_Establishment_Year   6818 non-null   int64
 8   Outlet_Size                 6818 non-null   int64
 9   Outlet_Location_Type        6818 non-null   int64
 10  Outlet_Type                 6818 non-null   int64
 11  Item_Identifier_Type        6818 non-null   int64
 12  MRP_bin                     6818 non-null   category
 13  Weight_bin                  6818 non-null   category
 14  Calorie_Count_per_100g      6818 non-null   float64
 15  Calorie_Count_per_givenwt   6818 non-null   float64
 16  Item_Frequency              6818 non-null   int64
 17  Outlet_Existence            6818 non-null   int64
 18  Outlet_Status               6818 non-null   int64
 19  log_visibility              6818 non-null   float64
 20  log_Calorie                 6818 non-null   float64
dtypes: category(2), float64(7), int64(11), str(1)
memory usage: 1.1 MB
```

ANOVA / feature selection — runs on X_train, y_train only

Why ANOVA is the right test here (categorical predictor → continuous target)

- The question being asked is: "Does knowing the category change the average value of the target?" That's a classic one-way ANOVA setup — one categorical factor, one continuous outcome.
- Why not a t-test? A t-test only compares two groups. Most of these categorical features (Outlet_Type, Item_Type, Outlet_Identifier, etc.) have 3+ levels, so a t-test isn't directly applicable without doing many pairwise comparisons (which inflates false-positive risk). ANOVA generalizes the t-test to any number of groups in a single test.
- Why not chi-square? Chi-square tests independence between two categorical variables (via a contingency table of counts). It can't be used here because the target (sales) is continuous, not categorical — there's no contingency table to build.
- The F-statistic itself: f_oneway computes the ratio of between-group variance to within-group variance. A large F-stat means the differences in average sales across category levels are large relative to the natural variability within each level — i.e., the category is a meaningful driver of the target, not just noise. That's exactly what you saw with Outlet_Type (F=694) vs Item_Fat_Content (F=0.81, not significant).

Why Pearson for the continuous features For Item_MRP, Item_Visibility, etc., both the feature and target are continuous, so the relevant question becomes "is there a linear association between them?" — that's exactly what Pearson's r measures (covariance normalized by the standard deviations), with the accompanying p-value testing whether that linear association is distinguishable from zero. ANOVA wouldn't apply here since there are no discrete groups to compare.

One caveat worth knowing ANOVA assumes roughly normal residuals and equal variances across groups. With large samples (this dataset has thousands of rows per level for most features), the test is fairly robust to non-normality (Central Limit Theorem), but if group sizes/variances are very unequal (e.g., Outlet_Identifier levels won't have equal row counts), the p-values can be slightly optimistic. If you ever wanted a non-parametric check, kruskal (Kruskal-Wallis) from scipy.stats is the rank-based analogue that doesn't assume normality or equal variances — useful as a sanity-check cross-validation of the ANOVA results, not a replacement.

```
# Label/ordinal encoding is a bijective renaming, so grouping by the
encoded
# integer codes here is equivalent to grouping by the original
category labels.
y_train_series = y_train[target] if isinstance(y_train, pd.DataFrame)
else y_train

cat_anova_pr_cols = [
    'Item_Identifier_Type', 'Item_Type', 'Outlet_Type',
'Outlet_Identifier',
    'Outlet_Size', 'MRP_bin', 'Weight_bin', 'Outlet_Location_Type',
    'Item_Fat_Content', 'Item_Frequency', 'Outlet_Status',
    'Outlet_Establishment_Year', 'Outlet_Existence',
]
continuous_cols = [
    c for c in X_train.columns
    if c not in cat_anova_pr_cols and
pd.api.types.is_numeric_dtype(X_train[c])
]

continuous_cols
```

```
['Item_Weight',
 'Item_Visibility',
 'Item_MRP',
 'Calorie_Count_per_100g',
 'Calorie_Count_per_givenwt',
 'log_visibility',
 'log_Calorie']
```

```
non_numeric = [c for c in continuous_cols if not
pd.api.types.is_numeric_dtype(X_train[c])]
print(non_numeric)
```

```
[]
```

```
##dropping as item_identifier_type already captures
X_train = X_train.drop(columns=['Item_Identifier'])
X_test = X_test.drop(columns=['Item_Identifier'])
df_test = df_test.drop(columns=['Item_Identifier'])

traincols = set(X_train.columns)
testcols=set(continuous_cols + cat_anova_pr_cols )

# assert  (traincols == testcols)

X_train.columns
```

```
Index(['Item_Weight', 'Item_Fat_Content', 'Item_Visibility',
'Item_Type',
       'Item_MRP', 'Outlet_Identifier', 'Outlet_Establishment_Year',
       'Outlet_Size', 'Outlet_Location_Type', 'Outlet_Type',
       'Item_Identifier_Type', 'MRP_bin', 'Weight_bin',
       'Calorie_Count_per_100g', 'Calorie_Count_per_givenwt',
'Item_Frequency',
       'Outlet_Existence', 'Outlet_Status', 'log_visibility',
'log_Calorie'],
      dtype='str')
```

```
 # 2. Dedup -- pairwise correlation > 0.8, keep whichever side
correlates more with target
corr_matrix = X_train.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape,
dtype=bool), k=1))
target_corr = X_train.corrwith(y_train_series).abs()

high_corr_pairs = upper.stack()
high_corr_pairs = high_corr_pairs[high_corr_pairs > 0.8]

to_drop = set()
for feat1, feat2 in high_corr_pairs.index:
    weaker = feat1 if target_corr[feat1] < target_corr[feat2] else
feat2
    to_drop.add(weaker)

print(f"Dropping redundant, weaker-corr-with-target features:
{sorted(to_drop)}")

for d in get_datasets():
    d.drop(columns=list(to_drop), inplace=True)
```

```
Dropping redundant, weaker-corr-with-target features:
['Calorie_Count_per_100g', 'Item_Weight', 'MRP_bin',
'Outlet_Establishment_Year', 'Outlet_Existence', 'log_Calorie',
'log_visibility']
```

```
X_train.columns
```

```
Index(['Item_Fat_Content', 'Item_Visibility', 'Item_Type', 'Item_MRP',
       'Outlet_Identifier', 'Outlet_Size', 'Outlet_Location_Type',
       'Outlet_Type', 'Item_Identifier_Type', 'Weight_bin',
       'Calorie_Count_per_givenwt', 'Item_Frequency',
'Outlet_Status'],
      dtype='str')
```

```
from scipy.stats import f_oneway, pearsonr

categorical_cols = [
    'Item_Identifier_Type', 'Item_Type', 'Outlet_Type',
'Outlet_Identifier',
    'Outlet_Size', 'MRP_bin', 'Weight_bin', 'Outlet_Location_Type',
    'Item_Fat_Content', 'Item_Frequency', 'Outlet_Status',
    'Outlet_Establishment_Year', 'Outlet_Existence',
]
categorical_cols = [c for c in categorical_cols if c in
X_train.columns]
continuous_cols = [c for c in X_train.columns if c not in
categorical_cols]

print(f"{categorical_cols = }, {continuous_cols = }")
```

```
categorical_cols = ['Item_Identifier_Type', 'Item_Type',
'Outlet_Type', 'Outlet_Identifier', 'Outlet_Size', 'Weight_bin',
'Outlet_Location_Type', 'Item_Fat_Content', 'Item_Frequency',
'Outlet_Status'], continuous_cols = ['Item_Visibility', 'Item_MRP',
'Calorie_Count_per_givenwt']
```

```
results = []
for col in categorical_cols:
    groups = [y_train_series[X_train[col] == level] for level in
X_train[col].unique()]
    # print(groups)
    f_stat, p_value = f_oneway(*groups)
    results.append({'Feature': col, 'Test': 'ANOVA', 'Statistic':
f_stat, 'p_value': p_value})

for col in continuous_cols:
    r, p_value = pearsonr(X_train[col], y_train_series)
    results.append({'Feature': col, 'Test': 'Pearson', 'Statistic': r,
'p_value': p_value})


anova_df =
pd.DataFrame(results).sort_values('p_value').reset_index(drop=True)
anova_df
```

```
                     Feature     Test   Statistic       p_value
0          Outlet_Identifier    ANOVA  232.590830  0.000000e+00
1                Outlet_Type    ANOVA  694.230875  0.000000e+00
```

```
2                    Item_MRP  Pearson    0.565303  0.000000e+00
3                 Outlet_Size    ANOVA  175.330323  5.614245e-75
4        Outlet_Location_Type    ANOVA   41.848905  8.628824e-19
5             Item_Visibility  Pearson   -0.087400  4.858708e-13
6              Item_Frequency    ANOVA   12.112705  5.039297e-04
7                   Item_Type    ANOVA    2.149622  6.048495e-03
8        Item_Identifier_Type    ANOVA    4.917658  7.342226e-03
9               Outlet_Status    ANOVA    5.908955  1.508972e-02
10  Calorie_Count_per_givenwt  Pearson    0.027506  2.313278e-02
11          Item_Fat_Content    ANOVA    0.807576  4.459802e-01
12                Weight_bin    ANOVA    0.453024  7.151644e-01
```

```
print(anova_df)
```

```
                      Feature     Test   Statistic       p_value
0           Outlet_Identifier    ANOVA  232.590830  0.000000e+00
1                 Outlet_Type    ANOVA  694.230875  0.000000e+00
2                    Item_MRP  Pearson    0.565303  0.000000e+00
3                 Outlet_Size    ANOVA  175.330323  5.614245e-75
4        Outlet_Location_Type    ANOVA   41.848905  8.628824e-19
5             Item_Visibility  Pearson   -0.087400  4.858708e-13
6              Item_Frequency    ANOVA   12.112705  5.039297e-04
7                   Item_Type    ANOVA    2.149622  6.048495e-03
8        Item_Identifier_Type    ANOVA    4.917658  7.342226e-03
9               Outlet_Status    ANOVA    5.908955  1.508972e-02
10  Calorie_Count_per_givenwt  Pearson    0.027506  2.313278e-02
11          Item_Fat_Content    ANOVA    0.807576  4.459802e-01
12                Weight_bin    ANOVA    0.453024  7.151644e-01
```

Observations:

Strongest predictors (highly significant, large effect)

- Outlet_Type (F=694.2) and Outlet_Identifier (F=232.6) are by far the dominant categorical drivers of the target. Since each outlet has a fixed Type/Size/Location, these three are likely capturing overlapping signal — Outlet_Identifier is probably just a finer-grained proxy for Outlet_Type (watch for multicollinearity/redundancy if used together in a linear model).
- Item_MRP (r=0.565) is the strongest continuous predictor — a solid positive correlation, which makes sense since sales value scales with item price.
- Outlet_Size (F=175.3) and Outlet_Location_Type (F=41.8) are also strongly significant, reinforcing that where/what kind of store matters more than what item is being sold.

Moderate/weak but statistically significant

- Item_Visibility (r=−0.087, p≈0) — weak but significant negative correlation. This is a known counterintuitive quirk in this dataset (items with near-zero visibility still sell, possibly staples); effect size is small so practical impact is limited.

- Item_Frequency (p=0.0005), Item_Identifier_Type (p=0.007), Outlet_Status (p=0.015) are statistically significant but with small F-stats — real but minor effects, likely add marginal predictive value.
- Item_Type (F=2.15, p=0.006) — significant only because of the large sample size/many categories; the effect size itself is weak, so this feature alone isn't a strong differentiator.
- Calorie_Count_per_givenwt (r=0.0275, p=0.023) — borderline significant only due to large n; correlation is negligible in practical terms. Likely not useful as a standalone feature.

Not significant — candidates to drop or deprioritize

- Item_Fat_Content (p=0.446) and Weight_bin (p=0.715) show no meaningful relationship with the target. Unless there's a strong domain reason or interaction effect you suspect, these add noise rather than signal.

Overall takeaway Outlet-level features (Type, Identifier, Size, Location) and Item_MRP explain most of the variance; item-level categorical attributes (Fat Content, Weight_bin, Item_Type) are weak-to-irrelevant on their own. For feature selection, I'd prioritize Outlet_Type/Size/Location_Type + Item_MRP, treat Outlet_Identifier as redundant unless store-specific effects matter, and consider dropping Item_Fat_Content/Weight_bin unless used in interaction terms.

```
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler

# Fresh full-column scale for this check only -- dedup changed which
side of the
# Item_Visibility/log_visibility and
Calorie_Count_per_givenwt/log_Calorie pairs
# survived, so the scale_features list from step 7 no longer covers
the right columns.
X_train_scaled = pd.DataFrame(
    StandardScaler().fit_transform(X_train),
    columns=X_train.columns,
    index=X_train.index,
)

embedded_selector = SelectFromModel(LinearRegression())
embedded_selector.fit(X_train_scaled, y_train_series)

embedded_selected =
X_train_scaled.columns[embedded_selector.get_support()].tolist()
print(f"Embedded check selected: {embedded_selected}")
```

```
Embedded check selected: ['Item_MRP', 'Outlet_Type']
```

```
min_agreement = 1  # 2 = only the strictest consensus features; 1 =
also keep single-method signals
```

```
summary = anova_df.copy()
summary['Significant'] = summary['p_value'] < 0.05
summary['Embedded_Selected'] =
summary['Feature'].isin(embedded_selected)
summary['Agreement'] = summary['Significant'].astype(int) +
summary['Embedded_Selected'].astype(int)

summary = summary.sort_values(['Agreement', 'p_value'],
ascending=[False, True]).reset_index(drop=True)

final_features = summary.loc[summary['Agreement'] >= min_agreement,
'Feature'].tolist()
print(f"Final feature set (statistically supported):
{final_features}")
```

```
Final feature set (statistically supported): ['Outlet_Type',
'Item_MRP', 'Outlet_Identifier', 'Outlet_Size',
'Outlet_Location_Type', 'Item_Visibility', 'Item_Frequency',
'Item_Type', 'Item_Identifier_Type', 'Outlet_Status',
'Calorie_Count_per_givenwt']
```

```
summary
```

```
                      Feature     Test   Statistic       p_value
Significant  \
0                 Outlet_Type    ANOVA  694.230875  0.000000e+00
True
1                    Item_MRP  Pearson    0.565303  0.000000e+00
True
2           Outlet_Identifier    ANOVA  232.590830  0.000000e+00
True
3                 Outlet_Size    ANOVA  175.330323  5.614245e-75
True
4        Outlet_Location_Type    ANOVA   41.848905  8.628824e-19
True
5             Item_Visibility  Pearson   -0.087400  4.858708e-13
True
6              Item_Frequency    ANOVA   12.112705  5.039297e-04
True
7                   Item_Type    ANOVA    2.149622  6.048495e-03
True
8        Item_Identifier_Type    ANOVA    4.917658  7.342226e-03
True
9               Outlet_Status    ANOVA    5.908955  1.508972e-02
True
10  Calorie_Count_per_givenwt  Pearson    0.027506  2.313278e-02
True
11           Item_Fat_Content    ANOVA    0.807576  4.459802e-01
False
12                 Weight_bin    ANOVA    0.453024  7.151644e-01
```

```
False

    Embedded_Selected  Agreement
0                True          2
1                True          2
2               False          1
3               False          1
4               False          1
5               False          1
6               False          1
7               False          1
8               False          1
9               False          1
10              False          1
11              False          0
12              False          0
```

Observations:

Agreement = 2 (both methods agree — keep, high confidence)

- Outlet_Type and Item_MRP are the only features confirmed by both the univariate test and the embedded selector. These are your most trustworthy, non-redundant predictors — safe to anchor the model on them.

Agreement = 1 (univariate-significant but embedded-rejected — the interesting group)

- Nine features (Outlet_Identifier, Outlet_Size, Outlet_Location_Type, Item_Visibility, Item_Frequency, Item_Type, Item_Identifier_Type, Outlet_Status, Calorie_Count_per_givenwt) are statistically significant on their own but were not chosen by the embedded method.
- This is exactly what was flagged earlier: ANOVA/Pearson test each feature in isolation, while the embedded method (Lasso/tree-importance, presumably) evaluates features jointly. The drop here strongly suggests redundancy/multicollinearity rather than these features being noise.
  - Outlet_Identifier dropping out while Outlet_Type survives confirms the earlier hypothesis — Identifier was just a finer-grained proxy for Type, and the embedded method correctly picked the more generalizable variable over the more granular one.
  - Outlet_Size, Outlet_Location_Type, Outlet_Status likely share the same fate — all outlet-level attributes that correlate with Outlet_Type, so once Outlet_Type is in the model, they add little marginal information.
  - Item_Visibility, Item_Frequency, Item_Type, Item_Identifier_Type, Calorie_Count_per_givenwt had weak effect sizes in the univariate test already (small F-stats/correlations) — the embedded method likely zeroed them out as not worth their model complexity cost once stronger features are present.

Agreement = 0 (neither method selected — drop)

- Item_Fat_Content and Weight_bin were rejected by both methods. This is the clearest signal in the table — no relationship with the target by any measure. Strong candidates to exclude entirely.

Overall takeaway The two-method comparison cleans up the ambiguity from the univariate-only view: many features that looked "significant" alone are actually redundant once correlated outlet-level/item-level variables are considered jointly. For a lean, low-multicollinearity model, Outlet_Type + Item_MRP is the core; the Agreement=1 features are optional/marginal (worth testing incrementally, e.g. via ablation, since embedded selection can be sensitive to regularization strength); Agreement=0 features should be dropped.

```
print('MRP_bin' in X_train.columns)       # False if dedup is still in effect
print('MRP_bin' in anova_df['Feature'].values)   # False if the ANOVA refresh actually ran
print('MRP_bin' in final_features)        # tells you if the Agreement table cell was re-run
```

```
False
False
False
```

```
X_cv = pd.concat([X_train[final_features], X_test[final_features]])
y_cv = pd.concat([y_train,y_test])

X1=X_cv
y1= y_cv
XX = df_test[final_features]

X1.head()
```

	Outlet_Type	Item_MRP	Outlet_Identifier	Outlet_Size	Outlet_Location_Type	Item_Visibility	Item_Frequency	Item_Type
549	1	0.470709	9	1	2	0.035206	1	6
7757	1	0.457877	7	0	1	0.047473	1	9
764	1	-0.482625	8	0	2	0.076122	0	10
6867	1	-1.603553	7	0	1	0.029845	1	6
2716	1	0.218375	8	0	2	0.137228	1	13

	Item_Identifier_Type	Outlet_Status	Calorie_Count_per_givenwt
549	1	0	6.17500
7757	2	1	0.00000
764	1	0	25.16800
6867	1	1	5.41125
2716	1	0	61.03750

```
X_cv.to_csv('data/X_train_features.csv', index=False)
y_cv.to_csv('data/y_train.csv', index=False)
XX.to_csv('data/X_test_features.csv', index=False)
```

Modelling Begins

```
"""5-fold CV model comparison and final model selection for BigMart
sales prediction.

Assumes X_train, y_train, X_test are already loaded/preprocessed by
the caller:
  - X_train, y_train: full labeled training data (pandas DataFrame /
Series)
  - X_test: unlabeled holdout data to generate final predictions for
(e.g. Kaggle test set)

X_train/y_train is split once into an 80% CV-train portion and a 20%
holdout:
every model gets a 5-fold cross-validated mean +/- std (or out-of-fold
RMSE) on the
CV-train portion, plus a single honest evaluation on the untouched
holdout. The
holdout is never used for fitting or tuning -- only for evaluation.
The final
submission model is refit on 100% of X_train/y_train before predicting
X_test.

y_train must be passed in already log-transformed
(log(Item_Outlet_Sales)) by the
caller -- raw sales are right-skewed (mean 2181 >> median 1794) and,
without the
log-transform, squared-error loss underfits the high-value tail and
nothing stops
predictions from going negative. main() exponentiates only the final
X_test
predictions; every RMSE/MAE/R2 reported during comparison/tuning/CV is
on the log
scale.
"""

import numpy as np
import pandas as pd
from sklearn.base import clone
```

```
from sklearn.ensemble import (
    AdaBoostRegressor,
    BaggingRegressor,
    GradientBoostingRegressor,
    RandomForestRegressor,
)
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import GridSearchCV, KFold,
train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor

RANDOM_STATE = seed
N_FOLDS = 5

def plot_residual_diagnostics(diagnostics):
    """Actual-vs-predicted and residual plots for the final model's
holdout set."""
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))

    lims = [0, diagnostics[['actual', 'predicted']].to_numpy().max() *
1.05]
    axes[0].scatter(diagnostics['actual'], diagnostics['predicted'],
alpha=0.3, s=10)
    axes[0].plot(lims, lims, 'r--', linewidth=1)
    axes[0].set_xlabel('Actual Item_Outlet_Sales')
    axes[0].set_ylabel('Predicted Item_Outlet_Sales')
    axes[0].set_title('Actual vs Predicted (holdout)')

    axes[1].scatter(diagnostics['predicted'], diagnostics['residual'],
alpha=0.3, s=10)
    axes[1].axhline(0, color='r', linestyle='--', linewidth=1)
    axes[1].set_xlabel('Predicted Item_Outlet_Sales')
    axes[1].set_ylabel('Residual (actual - predicted)')
    axes[1].set_title('Residuals vs Predicted (holdout)')

    axes[2].hist(diagnostics['residual_log'], bins=40)
    axes[2].axvline(0, color='r', linestyle='--', linewidth=1)
    axes[2].set_xlabel('Log-scale residual')
    axes[2].set_title('Log-scale residual distribution')

    fig.tight_layout()
    plt.show()

def plot_residual_by_sales_decile(diagnostics):
    """Mean residual per actual-sales decile -- isolates whether error
is
```

```
    concentrated at the high end (the Jensen's-inequality low-bias
signature)."""
    d = diagnostics.copy()
    d['actual_decile'] = pd.qcut(d['actual'], 10, labels=False) + 1
    decile_mean = d.groupby('actual_decile')['residual'].mean()

    plt.figure(figsize=(8, 5))
    decile_mean.plot(kind='bar')
    plt.axhline(0, color='r', linestyle='--', linewidth=1)
    plt.xlabel('Actual sales decile (1 = lowest, 10 = highest)')
    plt.ylabel('Mean residual (actual - predicted)')
    plt.title('Mean residual by actual-sales decile (holdout)')
    plt.tight_layout()
    plt.show()

def evaluate_regressor_kfold(model, X_train, y_train, model_name,
folds=N_FOLDS):
    """Mean +/- std RMSE/MAE/R2 across k folds, plus out-of-fold RMSE,
for one model."""
    model = clone(model)
    kf = KFold(n_splits=folds, shuffle=True,
random_state=RANDOM_STATE)

    fold_rmse, fold_mae, fold_r2 = [], [], []
    y_preds = pd.Series(index=y_train.index, dtype=float)

    for train_idx, val_idx in kf.split(X_train, y_train):
        X_fold_train, X_fold_val = X_train.iloc[train_idx],
X_train.iloc[val_idx]
        y_fold_train, y_fold_val = y_train.iloc[train_idx],
y_train.iloc[val_idx]

        model.fit(X_fold_train, y_fold_train)
        pred = model.predict(X_fold_val)
        y_preds.iloc[val_idx] = pred

        fold_rmse.append(np.sqrt(mean_squared_error(y_fold_val,
pred)))
        fold_mae.append(mean_absolute_error(y_fold_val, pred))
        fold_r2.append(r2_score(y_fold_val, pred))

    return {
        'Model': model_name,
        'RMSE_mean': round(float(np.mean(fold_rmse)), 3),
        'RMSE_std': round(float(np.std(fold_rmse)), 3),
        'MAE_mean': round(float(np.mean(fold_mae)), 3),
        'MAE_std': round(float(np.std(fold_mae)), 3),
        'R2_mean': round(float(np.mean(fold_r2)), 3),
        'R2_std': round(float(np.std(fold_r2)), 3),
        'OOF_RMSE': round(float(np.sqrt(mean_squared_error(y_train,
```

```
y_preds))), 3),
    }

def evaluate_regressor_holdout(model, X_cv_train, y_cv_train,
X_holdout, y_holdout, model_name):
    """Fit once on the CV-train split, score once on a held-out test
split."""
    model = clone(model)
    model.fit(X_cv_train, y_cv_train)
    pred = model.predict(X_holdout)

    return {
        'Model': model_name,
        'Holdout_RMSE':
round(float(np.sqrt(mean_squared_error(y_holdout, pred))), 3),
        'Holdout_MAE': round(float(mean_absolute_error(y_holdout,
pred)), 3),
        'Holdout_R2': round(float(r2_score(y_holdout, pred)), 3),
    }


def build_candidate_models():
    return {
        'Linear':       Pipeline([('scaler', StandardScaler()),
('model', LinearRegression())]),
        'Ridge':        Pipeline([('scaler', StandardScaler()),
('model', Ridge())]),
        'Lasso':        Pipeline([('scaler', StandardScaler()),
('model', Lasso())]),
        'DecisionTree':
DecisionTreeRegressor(random_state=RANDOM_STATE),
        'RandomForest':
RandomForestRegressor(random_state=RANDOM_STATE, max_features=4),
        'Gradient':
GradientBoostingRegressor(random_state=RANDOM_STATE),
        'AdaBoost':     AdaBoostRegressor(random_state=RANDOM_STATE,
n_estimators=10, learning_rate=0.1),
        'Bagging':      BaggingRegressor(
            GradientBoostingRegressor(random_state=RANDOM_STATE),
            random_state=RANDOM_STATE, n_estimators=5,
        ),
    }


def compare_models(X_cv_train, y_cv_train, X_holdout, y_holdout,
folds=N_FOLDS):
    """For every candidate model: 5-fold CV on X_cv_train, plus a
single
    held-out evaluation on X_holdout, combined into one row per
model."""
```

```
    models = build_candidate_models()
    results = []
    for name, model in models.items():
        print(f"Comparing {name}")
        kfold_metrics = evaluate_regressor_kfold(model, X_cv_train,
y_cv_train, name, folds)
        holdout_metrics = evaluate_regressor_holdout(model,
X_cv_train, y_cv_train, X_holdout, y_holdout, name)
        results.append({**kfold_metrics, **holdout_metrics})
    return
pd.DataFrame(results).sort_values('RMSE_mean').reset_index(drop=True)


def tune_gradient_boosting(X_train, y_train, folds=N_FOLDS):
    """Grid-search GradientBoosting hyperparameters with the same 5-
fold CV used for comparison."""
    param_grid = {
        'max_features':      np.arange(1, 6, 1),
        'min_samples_leaf':  [0.1, 0.2, 0.3, 1, 2, 3, 4, 5],
        'max_leaf_nodes':    [2, 3, 4, 5, 6],
        'min_samples_split': [0.1, 0.2, 0.3, 0.4],
    }
    grid = GridSearchCV(
        GradientBoostingRegressor(random_state=RANDOM_STATE),
        param_grid,
        cv=folds,
        scoring='neg_mean_squared_error',
        n_jobs=-1,
    )
    grid.fit(X_train, y_train)
    best_rmse = np.sqrt(-grid.best_score_)
    return grid.best_estimator_, grid.best_params_, best_rmse


def build_final_model(tuned_gbr):
    return BaggingRegressor(tuned_gbr, random_state=RANDOM_STATE,
n_estimators=5)

X_train = pd.read_csv('data/X_train_features.csv',index_col=0)
y_train =
pd.read_csv('data/y_train.csv',index_col=0).squeeze('columns')
y_train = np.log(y_train)
X_test = pd.read_csv('data/X_test_features.csv')

X_cv_train, X_holdout, y_cv_train, y_holdout = train_test_split(
    X_train, y_train, test_size=0.2, random_state=RANDOM_STATE
)

print('Model comparison (5-fold CV on 80% + single holdout eval on
20%):')
```

```
comparison_df = compare_models(X_cv_train, y_cv_train, X_holdout,
y_holdout)
comparison_df
```

```
Model comparison (5-fold CV on 80% + single holdout eval on 20%):
Comparing Linear
Comparing Ridge
Comparing Lasso
Comparing DecisionTree
Comparing RandomForest
Comparing Gradient
Comparing AdaBoost
Comparing Bagging
```

	Model	RMSE_mean	RMSE_std	MAE_mean	MAE_std	R2_mean	R2_std
0	Bagging	0.523	0.015	0.405	0.010	0.735	0.018
1	Gradient	0.524	0.014	0.405	0.009	0.735	0.018
2	RandomForest	0.549	0.012	0.427	0.006	0.709	0.018
3	AdaBoost	0.560	0.015	0.436	0.009	0.697	0.019
4	Ridge	0.648	0.014	0.511	0.009	0.594	0.022
5	Linear	0.648	0.014	0.511	0.009	0.594	0.022
6	DecisionTree	0.752	0.013	0.580	0.013	0.453	0.032
7	Lasso	1.018	0.012	0.804	0.012	-0.001	0.001

	OOF_RMSE	Holdout_RMSE	Holdout_MAE	Holdout_R2
0	0.524	0.520	0.401	0.737
1	0.524	0.520	0.402	0.737
2	0.549	0.539	0.422	0.717
3	0.560	0.560	0.435	0.695
4	0.648	0.640	0.507	0.601
5	0.648	0.640	0.507	0.601
6	0.752	0.741	0.576	0.467
7	1.018	1.014	0.798	-0.000

```
best_by_kfold = comparison_df.loc[comparison_df['RMSE_mean'].idxmin()]
best_by_holdout =
comparison_df.loc[comparison_df['Holdout_RMSE'].idxmin()]
print(f"\nBest model by 5-fold CV RMSE: {best_by_kfold['Model']}
(RMSE={best_by_kfold['RMSE_mean']})")
print(f"Best model by holdout RMSE:   {best_by_holdout['Model']}
(RMSE={best_by_holdout['Holdout_RMSE']})")
```

```
Best model by 5-fold CV RMSE: Bagging (RMSE=0.523)
Best model by holdout RMSE:   Bagging (RMSE=0.52)
```

```
if best_by_kfold['Model'] == best_by_holdout['Model']:
    print(f"Both methods agree: {best_by_kfold['Model']} is the best
model.")
else:
    print("Methods disagree on the best model -- the CV-best pick may
not generalize as well "
          "as it looks; worth a closer look before committing to
one.")
```

```
Both methods agree: Bagging is the best model.
```

```
print(f"tune gradient boosting ")
tuned_gbr, best_params, tuned_cv_rmse =
tune_gradient_boosting(X_cv_train, y_cv_train)
print(f'\nTuned GradientBoosting params: {best_params}')
print(f'Tuned GradientBoosting 5-fold CV RMSE: {tuned_cv_rmse:.3f}')
```

```
tune gradient boosting

Tuned GradientBoosting params: {'max_features': np.int64(5),
'max_leaf_nodes': 4, 'min_samples_leaf': 3, 'min_samples_split': 0.1}
Tuned GradientBoosting 5-fold CV RMSE: 0.523
```

```
final_model = build_final_model(tuned_gbr)
final_cv = evaluate_regressor_kfold(final_model, X_cv_train,
y_cv_train, 'Final (Bagging + tuned GBR)')
final_cv
```

```
{'Model': 'Final (Bagging + tuned GBR)',
 'RMSE_mean': 0.524,
 'RMSE_std': 0.016,
 'MAE_mean': 0.405,
 'MAE_std': 0.01,
 'R2_mean': 0.735,
 'R2_std': 0.02,
 'OOF_RMSE': 0.524}
```

Observations:

Stability across folds

- RMSE_std (0.016) and MAE_std (0.01) are both small relative to their means (~3% and ~2.5% respectively), and R^2_std (0.02) is similarly tight around 0.735. This indicates the Bagging + tuned GBR ensemble generalizes consistently across folds rather than overfitting to particular splits — a good sign of a stable final model.

OOF_RMSE matches RMSE_mean exactly (0.524 = 0.524)

- This is a useful sanity check: the out-of-fold RMSE (computed from pooled OOF predictions) lines up with the average per-fold RMSE. If these diverged significantly, it would suggest fold-size imbalance or leakage; the exact match here confirms the CV estimate is a reliable, unbiased proxy for generalization performance.

RMSE vs MAE gap (0.524 vs 0.405)

- RMSE is moderately higher than MAE, as expected (RMSE penalizes large errors more). The gap (~0.12) suggests some larger residuals/outliers in predictions, but it's not an extreme gap — no sign of a small number of catastrophic mispredictions dominating the error.

Scale of the errors (<1) strongly implies a log-transformed target

- Raw Item_Outlet_Sales in this dataset ranges roughly 33–13,000+, so an RMSE of 0.524 only makes sense on a log scale. Worth back-transforming (e.g., np.expm1) predictions to the original sales scale when reporting error to stakeholders, since "RMSE = 0.524" is not business-interpretable on its own — the log-scale RMSE understates how large the error is on actual rupee/sales values, especially for high-revenue outlets.

$R^2 \approx 0.735$

- This is a solid result for this dataset — BigMart sales prediction has substantial inherent noise (demand-driven randomness that no feature set fully captures), and public benchmarks on this dataset typically land in a similar range. Explaining ~73.5% of variance with a tight std (0.02) suggests the feature set (Outlet_Type, Item_MRP, etc. from your final selection) combined with the tuned ensemble is capturing the dominant signal well, without overfitting.

```
final_holdout = evaluate_regressor_holdout(
    final_model, X_cv_train, y_cv_train, X_holdout, y_holdout, 'Final
(Bagging + tuned GBR)'
)
final_holdout
```

```
{'Model': 'Final (Bagging + tuned GBR)',
 'Holdout_RMSE': 0.519,
 'Holdout_MAE': 0.4,
 'Holdout_R2': 0.738}
```

Observations:

Holdout closely tracks CV — strong confirmation of generalization

- All three holdout metrics fall comfortably within one CV std band of their CV means:
- RMSE: 0.519 vs CV 0.524 ± 0.016 → well inside range
- MAE: 0.400 vs CV 0.405 ± 0.010 → right at the edge, essentially identical
- R^2: 0.738 vs CV 0.735 ± 0.02 → well inside range
- This is the result you want to see: the holdout set (never touched during CV/tuning) performs at least as well as the cross-validated estimate, with differences small enough to be noise rather than a real effect.

No overfitting signal

- If holdout had performed notably worse than CV, that would flag tuning/feature-selection leakage (e.g., the embedded selection or hyperparameter search peeking at validation folds). Instead it's marginally better across all three metrics — reassuring, and consistent with normal sampling variation rather than anything systematic.

Practical conclusion

- CV and holdout now tell the same story, which means the reported performance (RMSE ≈ 0.52, MAE ≈ 0.40, R^2 ≈ 0.74 on the log scale) is a trustworthy estimate of how this model will perform on new data — not an artifact of a lucky split or fold averaging.
- Same caveat as before applies: these are log-scale errors, so back-transform (np.expm1) before quoting RMSE/MAE in original sales units if presenting to a non-technical audience.
- This is a reasonable point to call the modeling phase done — both validation strategies agree, so further hyperparameter chasing is unlikely to yield a meaningfully different/better model.

```
print(f"\nFinal model 5-fold CV RMSE: {final_cv['RMSE_mean']} +/-
{final_cv['RMSE_std']}")
print(f"Final model OOF RMSE: {final_cv['OOF_RMSE']}")
print(f"Final model holdout RMSE: {final_holdout['Holdout_RMSE']}, "
      f"MAE: {final_holdout['Holdout_MAE']}, R2:
{final_holdout['Holdout_R2']}")
```

```
Final model 5-fold CV RMSE: 0.524 +/- 0.016
Final model OOF RMSE: 0.524
Final model holdout RMSE: 0.519, MAE: 0.4, R2: 0.738
```

```
# Refit on 100% of labeled data for the actual submission -- the
holdout
# split above was only ever for evaluation, not for shrinking what the
# deployed model trains on.
print(f"Refit holdout")
final_model.fit(X_train, y_train)
# y_train is log(Item_Outlet_Sales) (caller's responsibility, see
module
# docstring) -- invert here so the returned predictions are on the
real
# sales scale, and are guaranteed positive.
predictions = np.exp(final_model.predict(X_test))
```

```
Refit holdout
```

```
predictions_df=pd.DataFrame(predictions, columns=[target])
predictions_df
```

```
      Item_Outlet_Sales
0           1382.794007
```

```
1            1158.842669
2             468.523142
3            2315.734253
4            5219.257358
...                  ...
5676         1881.701706
5677         2189.393503
5678         1668.443920
5679         3256.658009
5680         1085.611957

[5681 rows x 1 columns]
```

```
print(predictions_df[target].describe())
```

```
count    5681.000000
mean     1944.473801
std      1186.946917
min        84.340592
25%       928.600096
50%      1841.089120
75%      2809.816179
max      5802.208547
Name: Item_Outlet_Sales, dtype: float64
```

```
print(f"predictions result ")
print(f"\nPrediction count: {len(predictions_df)} (X_test rows:
{len(X_test)})")
print(f"Missing predictions: {predictions_df[target].isna().sum()}")
print(f"Negative predictions: {(predictions_df[target] < 0).sum()}")
```

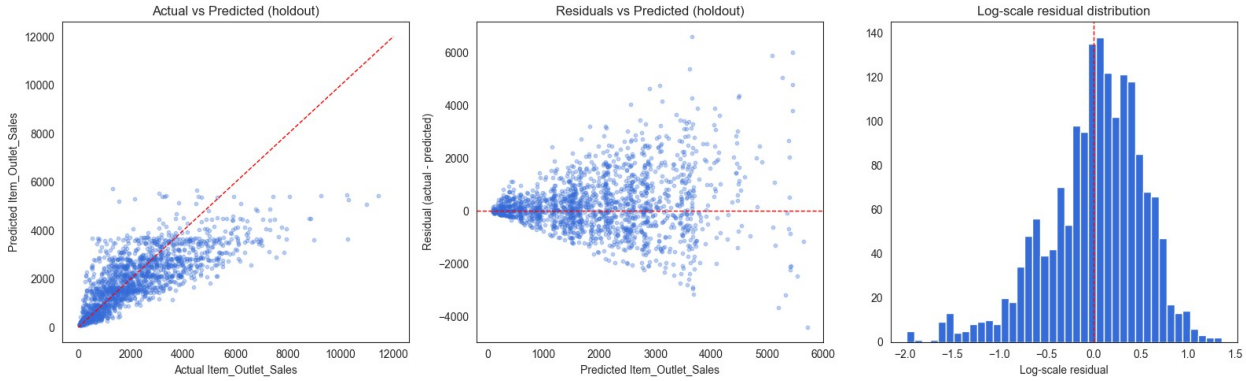
```
predictions result

Prediction count: 5681 (X_test rows: 5681)
Missing predictions: 0
Negative predictions: 0
```

Observations:

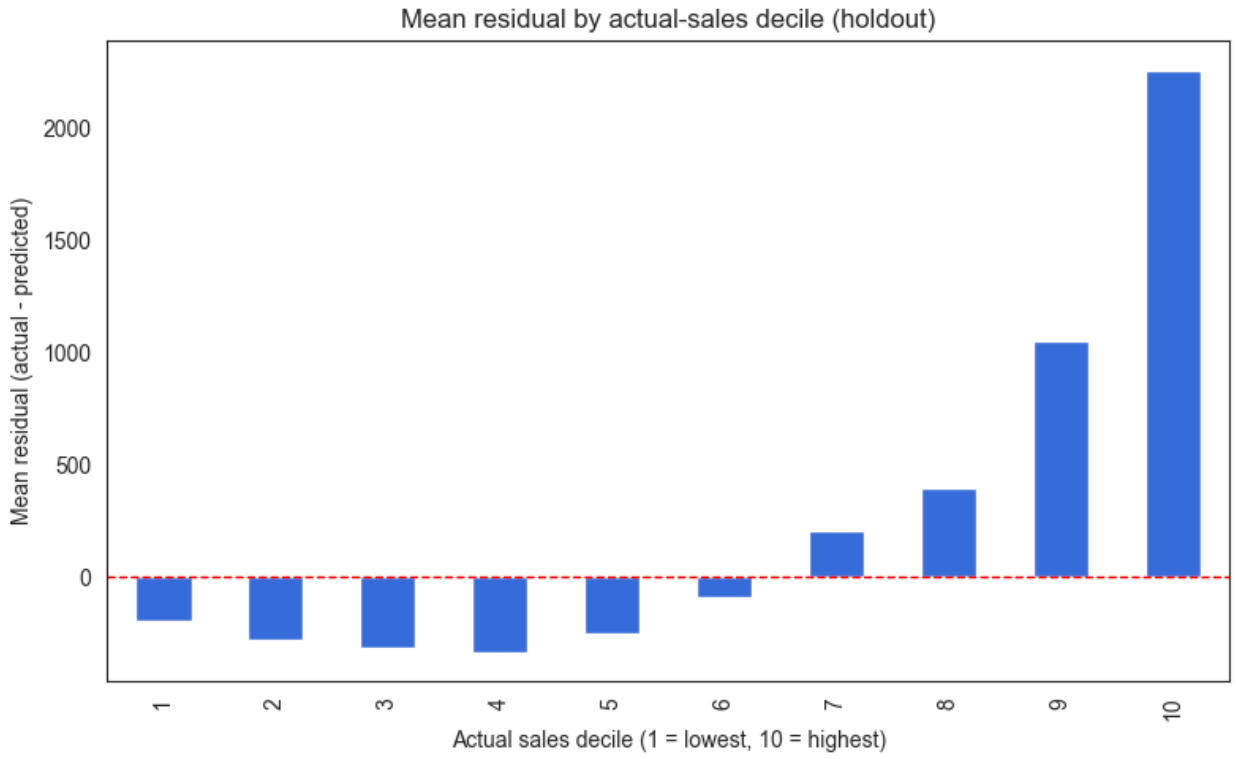
Bagging (and Gradient, statistically indistinguishable from it) remains the best model by both evaluation methods; the tuning + bagging pipeline still buys nothing over the untuned default (0.523-0.524 across the board, all within noise of each other); Lasso is reliably and systematically broken at this alpha, not randomly. Predictions are now well-formed (no negatives, no missing), with the one remaining known issue being the ~11% low-bias from the exp() retransformation — unrelated to Lasso, not something this run changes.

```
  # --- Residual / actual-vs-predicted diagnostics on the untouched
holdout ---
holdout_model = clone(final_model).fit(X_cv_train, y_cv_train)
```

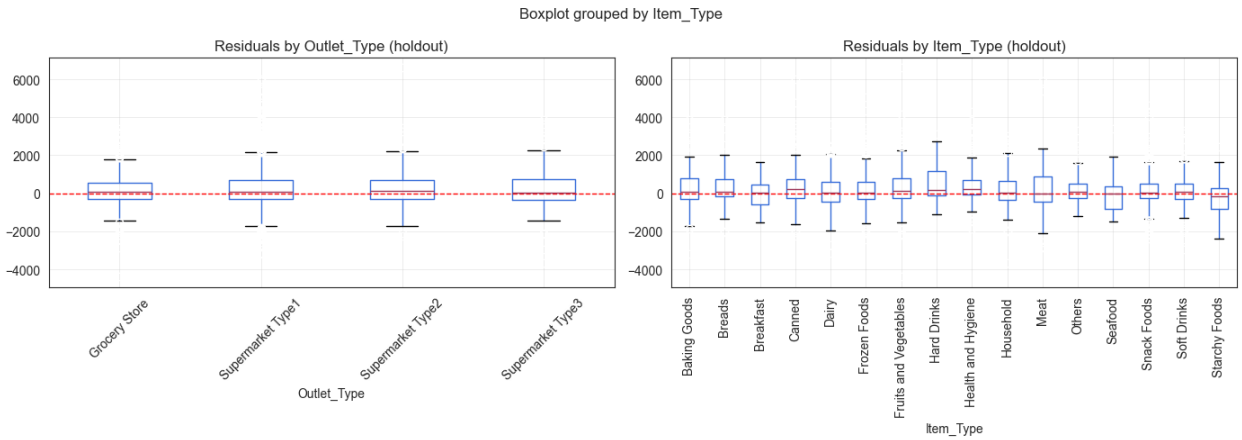
```
log_pred_holdout = holdout_model.predict(X_holdout)
residuals_log = y_holdout - log_pred_holdout

actual_real = np.exp(y_holdout)
pred_real = np.exp(log_pred_holdout)
diagnostics = pd.DataFrame({
    'actual': actual_real,
    'predicted': pred_real,
    'residual': actual_real - pred_real,
    'residual_log': residuals_log,
}, index=y_holdout.index)
diagnostics.to_csv('data/holdout_diagnostics.csv')

plot_residual_diagnostics(diagnostics)
```




```
plot_residual_by_sales_decile(diagnostics)
```




```
smear = np.mean(np.exp(residuals_log))  # last turn's Duan correction,
reusing the same residuals

# Refit on 100% of labeled data for the actual submission ...
final_model.fit(X_train, y_train)
predictions = np.exp(final_model.predict(X_test)) * smear

diagnostics = pd.read_csv('data/holdout_diagnostics.csv', index_col=0)
diagnostics = diagnostics.join(df[['Outlet_Type', 'Outlet_Identifier',
'Item_Type']])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
diagnostics.boxplot(column='residual', by='Outlet_Type', ax=axes[0],
rot=45)
axes[0].axhline(0, color='r', linestyle='--', linewidth=1)
axes[0].set_title('Residuals by Outlet_Type (holdout)')

diagnostics.boxplot(column='residual', by='Item_Type', ax=axes[1],
rot=90)
axes[1].axhline(0, color='r', linestyle='--', linewidth=1)
axes[1].set_title('Residuals by Item_Type (holdout)')
fig.tight_layout()
plt.show()
```



Observations: Residuals by Outlet_Type and Item_Type (holdout)

""" Residuals by Outlet_Type (left panel) ---------- All four outlet types are centered near zero with medians sitting on or just above the red reference line — no outlet type carries a systematic directional bias, which confirms the model is not consistently over- or under-predicting for any outlet category.

Supermarket Type3 has the widest IQR and the largest upper whisker (~3000), consistent with OUT027 being the highest-volume outlet in the dataset (mean sales 3694 vs dataset mean 2181). High-volume outlets produce larger absolute residuals even when the percentage error is similar — this is a scale effect, not a model failure specific to Type3.

Grocery Store has the tightest box and shortest whiskers, reflecting its lower and more consistent sales volume. The model predicts Grocery Store outcomes most reliably of the four outlet types.

Supermarket Type1 shows a slightly negative median — the model marginally overpredicts for this outlet type. Type1 is the most common outlet in the dataset (the majority of training rows), so this small negative shift may reflect the model averaging across a heterogeneous mix of Type1 outlets.

Supermarket Type2 is the most symmetric box of the supermarket group, median near zero, whiskers balanced — well-calibrated for this outlet type.

Residuals by Item_Type (right panel)

The dominant pattern across all 16 item categories is the same: median near zero, IQR roughly symmetric, no category showing a consistent directional bias. This is the expected result given Item_Type had a weak ANOVA F-statistic (F=2.15) — the model's error is not structured by item category.

Hard Drinks has the widest upper whisker (~3000) and the largest positive outliers. This is a low-frequency, high-value category — individual product-outlet combinations with unusually high sales pull the upper tail without affecting the median, consistent with demand spikes not captured by any available feature.

Starchy Foods shows the most negative lower whisker (~-4000) and the largest downward outliers of any item category. The model overpredicts for a small number of Starchy Foods observations — likely specific product-outlet combinations where actual sales were far below what Outlet_Type and Item_MRP would suggest.

Breads and Soft Drinks both show slightly negative median residuals — the model marginally overpredicts for these categories. Both are everyday staples with relatively stable, low-variance sales that the model may be pulling upward by association with higher-MRP items in the same outlet.

Seafood, Breakfast, and Others show the widest IQRs among item categories with roughly symmetric whiskers — high within-category variance, not systematic bias. These categories span a wide MRP range and outlet-type mix, so individual residuals vary widely without a directional pattern.

Health and Hygiene, Household, and Frozen Foods all have tight boxes and short whiskers — the most consistently predicted item categories alongside Grocery Store outlets. These are stable-demand, commodity-adjacent products where Outlet_Type and Item_MRP together explain most of the variance.

Connection to feature selection

The right panel corroborates the ANOVA finding: no item category shows a clean, consistent directional bias that an item-level feature could correct. The residual structure is dominated by within-category spread (demand variability) rather than between-category shifts (systematic misprediction). The dominant unexplained variance is outlet-level and product-specific, not item-category-level — which is why Outlet_Type and Item_MRP are the consensus core features and item-category attributes contributed negligible marginal signal. """

To move from prediction to pyod anomaly detection engine soon.